

Evidence-Grounded Explanations for Log-Based Anomaly Detection: A Screener–Reasoner Framework with Automated Evidence Verification

Chi-Ming Chou (corresponding author), Shang-Kuan Chen

Yuan Ze University, No. 135, Yuandong Rd., Zhongli Dist., Taoyuan City 320315,
Taiwan (R.O.C.)

E-mail addresses: s1136107@mail.yzu.edu.tw (Chi-Ming Chou),

cotachen@saturn.yzu.edu.tw (Shang-Kuan Chen)

Keywords:

Log anomaly detection

Evidence-grounded explanation

Retrieval-augmented generation

Automated evidence verification

Abstract

Log-based anomaly detection is essential for maintaining the reliability of large-scale computing systems. Deep learning models can detect anomalous log sessions with near-ceiling accuracy, but they still fail at the step that matters most to operators: producing trustworthy, actionable explanations that are explicitly grounded in evidence, verifiable, and scalable for operational cost control. This work proposes an evidence-centric Screener-Reasoner framework that decouples a lightweight anomaly screener from a retrieval-augmented LLM reasoner. The Screener, a parser-free model built on linear self-attention that preserves the full raw log context while enabling efficient inference on long sessions, flags anomalies and routes to the Reasoner. The Reasoner receives similar historical evidence and outputs structured trace JSON in which each claim must reference evidence identifiers. An automated evidence verifier enforces schema validity, citation integrity, span consistency, and grounding constraints. The framework adds confidence-based gating to study cost trade-offs under bounded LLM budgets. Extensive experiments on BGL and HDFS show that the proposed framework preserves strong detection performance while delivering highly structured, evidence-grounded explanations with high verification pass rates. Human evaluation confirms strong correctness, completeness, and actionability. These results establish a practical path from accurate log anomaly detection to auditable, cost-aware, and deployable explanations in production environments.

1. INTRODUCTION

Interconnected large-scale systems have become essential to modern infrastructure, enabling smooth data sharing, remote access, and intelligent coordination across various environments. As these systems grow in size and complexity, the security attack surface also increases considerably. Failing to detect and address anomalies and vulnerabilities can lead to operational disruptions, financial losses, data breaches, legal and regulatory penalties, reputational damage, and customer loss. For example, credential stuffing attacks on Roku [1] compromised over 576,000 accounts. Likewise, the Target data breach incident [2] affected over 40 million credit and debit card accounts, and 70 million personal records were stolen. System administrators must promptly detect and alert to anomalous events to prevent those incidents.

Software developers record and print system status in the system logs shown in Figure 1, which typically include a timestamp, logging level, and log messages. System logs are crucial for identifying abnormal events that may indicate security breaches, system failures, or performance issues. Yet the scale, velocity, and heterogeneity of contemporary log streams make manual inspection slow, expensive, and operationally unsustainable. This has made automated log analytics a core requirement for dependable computing and large-scale service operations.

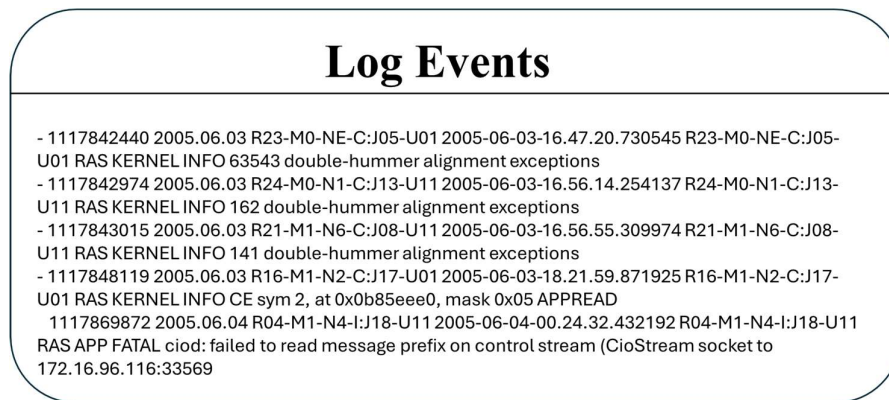


Figure 1. System logs include timestamps, logging levels, and messages.

Within automated log analytics, log-based anomaly detection has emerged as a key mechanism for identifying abnormal execution sessions from raw system logs. A typical pipeline usually involves these steps. First, raw logs are processed, either with or without a log parser, and grouped into log sessions. The second step converts these sessions into vectors or tokens to prepare them for anomaly detection. The final step classifies each log session as either normal or anomalous. When effective, such models reduce the monitoring burden, accelerate incident triage, and improve service reliability. Consequently, log-based anomaly detection has received substantial attention in software engineering and Deep Learning research. Chen et al. [3] classified successful and failed requests using a decision tree. Breier et al. [4] utilized PCA to analyze and highlight anomalies. LogCluster [5] segmented logs through clustering and performed anomaly detection using an unsupervised learning method. Machine learning methods struggle to generalize because they cannot effectively capture sequential or contextual relationships in high-dimensional log spaces. As a result, researchers shifted their focus to deep learning techniques. DeepLog [6] used an LSTM (Long Short-Term Memory) to capture the sequential relationships among log templates, achieving high accuracy in anomaly detection. Lu et al. [7] employed a Convolutional Neural Network (CNN) to learn the hidden relationships among log events. Transformers [8], which combine efficient parallel processing with the Multi-headed Self-attention mechanism, outperform LSTMs, which are inherently sequential in their ability to capture both local and long-range dependencies. Compared to CNNs, which focus on regional patterns and require depth to understand global context, Transformers analyze log sequences more effectively. LogBERT [9] leverages the Bidirectional Encoder Representations from Transformers (BERT) model to learn patterns in normal log sequences through unsupervised learning and then identifies outliers. LogGPT [10] employed the Generative Pre-trained Transformer (GPT) model to detect log anomalies by analyzing

log sequences with Chain-of-Thought (CoT) prompts. FSMFLog [11] used SBERT to obtain semantic features as input, also projected the time and type features from parsed log event sequences, inserting them into the Transformer encoder as biases for anomaly detection. LogFormer [12] used a Transformer-based framework for log anomaly detection, thereby improving detection by incorporating overlooked temporal and other kinds of features during log parsing. Recent work pushes benchmark detection performance close to saturation on widely used datasets.

A large portion of the literature still depends on log parsing to convert raw messages into structured templates or event sequences before modeling. In the system logs, developers use unrestricted text entries, such as variable names, abbreviations, or technical terms, to record status updates or errors. These unseen text combinations lead to out-of-vocabulary (OOV) issues. Additionally, system logs are unstructured text, making them difficult to analyze automatically. To address these issues in anomaly detection, as illustrated in Figure 2, prior work used log parsers [13] to extract timestamps, logging levels, components, events, and parameters from logs, transforming raw logs into structured event templates. Event templates are created by retaining keywords that are constant in logs and removing parameters that carry dynamic runtime information. Existing log parsing methods include language modeling [14], heuristics [15–17], frequent pattern mining [18,19], clustering [20], and longest common subsequence [21]. Among them, Drain [16] is recognized as the most accurate and efficient [13]. Although this preprocessing step simplifies downstream learning, it also introduces a fundamental bottleneck in representation. Parsing can remove dynamic runtime details, merge semantically distinct events, split semantically similar events, and require continual maintenance as logging statements evolve. Studies [22,23] have shown that computational overhead from log parsing, semantic misunderstanding, and the loss of contextual details can negatively impact the performance of log anomaly

detection models. For robust anomaly analysis, preserving raw-log context is therefore not a convenience but a modeling requirement.

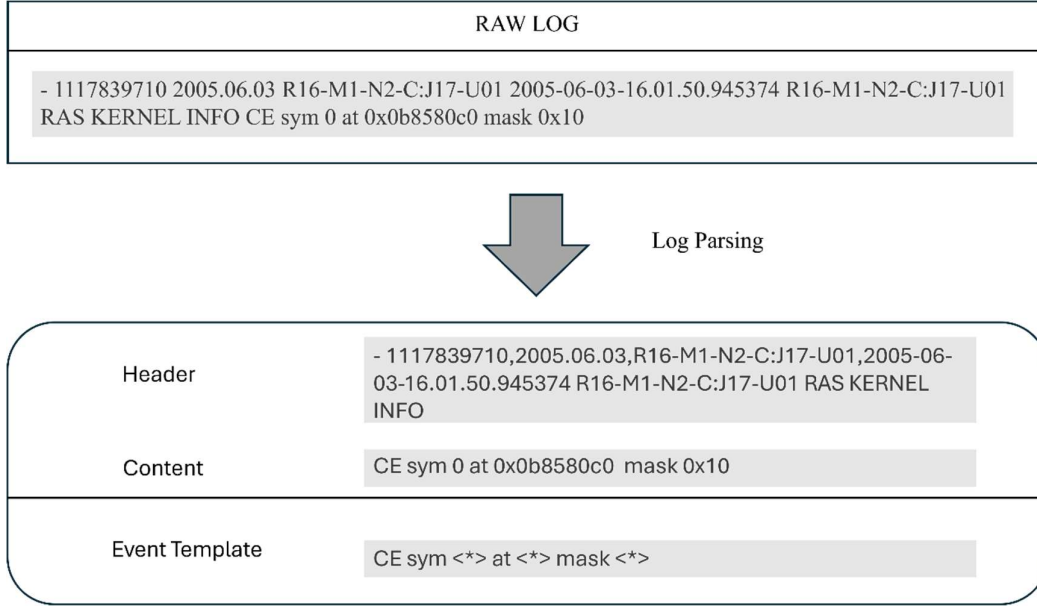


Figure 2. An example of the log parsing process

Direct raw-log modeling introduces a second challenge: sequence length. When all contextual information is retained, session-level inputs can easily reach thousands of tokens. Under such conditions, conventional Transformer self-attention becomes computationally costly because its complexity scales quadratically with sequence length. In practice, this often leads to truncation, compression, or selective feature removal, undermining contextual accuracy. A viable raw-log detector must therefore both avoid parser-induced information loss and be computationally efficient on long sessions.

To mitigate the drawbacks of using a log parser, preserve all contextual information in logs, and efficiently detect anomalies, this work proposes AllLinLog (utilizing all contextual information with a linear self-attention encoder for log-based anomaly detection) as the Screener. AllLinLog is a parser-free anomaly detector that

applies GPT-based byte pair encoding to raw log sessions and uses linear self-attention to model long-range dependencies with lower computational overhead than standard self-attention. This design preserves complete session context without template extraction, making long-session inference practical. The proposed detector is not positioned as an isolated endpoint but as the screening foundation of a larger evidence-grounded anomaly analysis system. Its purpose is to deliver strong, scalable anomaly screening over raw logs so that subsequent reasoning operates on a reliable detection front end.

Accurate screening alone does not satisfy operational needs. A binary anomaly label does not explain why a session was flagged, which lines support the decision, or how the observed behavior relates to known failure patterns. Operations need interpretable, auditable, and sufficiently grounded explanations to support diagnosis, escalation, and post-incident review. This requirement exposed a major gap in the current literature. Although detection performance approaches saturation on benchmark datasets, explanations for anomalies remain underdeveloped. LLM-generated explanations are often fluent but lack adequate grounding in verifiable evidence. Without traceable support, such explanations are hard to trust in critical operational workflows.

To address this gap, this article proposes a unified Screener-Reasoner framework for evidence-grounded log anomaly analysis, as shown in Figure 3. The framework combines a linear self-attention-based screener with a retrieval-augmented LLM reasoner that produces structured explanations for anomalous sessions. The Reasoner retrieves support from a train-only evidence store, generates explanations under a fixed trace schema, and links claims to explicit evidence identifiers. The framework incorporates automated evidence verification to ensure schema validity, citation accuracy, span consistency, and grounding constraints, thereby enhancing overall

reliability. Furthermore, confidence-based gating is employed to regulate the activation of explanations, thereby allowing the system to operate within constrained inference budgets rather than assuming unlimited access to LLMs.

This article is guided by three research questions. First, to what extent can the Screener-Reasoner framework produce traceable, evidence-grounded explanations for anomalous sessions, as measured by structured output compliance, evidence citation coverage, and human-evaluated explanation quality? Second, what is the contribution of retrieval-augmented generation (RAG) to the evidence grounding quality of LLM-produced anomaly explanations? Third, under a fixed LLM inference budget, how do confidence-based gating strategies affect the trade-off between explanation coverage, per-session quality, and operational cost, as measured by trigger rate, token consumption, and explanation quality metrics? These questions deliberately shift the focus from incremental gains in anomaly classification accuracy to the broader problem of building an explainable, auditable, and practically deployable log anomaly analysis framework.

The contributions of this article are fourfold. First, it formulates log anomaly analysis as a unified screener-reasoner problem that decouples efficient anomaly screening from evidence-grounded explanations, thereby invoking the costly LLM reasoner only on screener-flagged anomalies. Second, it employs a parser-free, linear, self-attention-based screener that operates directly on raw log tokens, preserving full lexical context while scaling to long sessions without parsing logs. Third, it presents a structured trace schema and an automated verification pipeline with multiple checks that enforce claim-level traceability. Ablation studies show that retrieval-augmented generation is the key mechanism for grounding cross-referential evidence. Fourth, it evaluates confidence-based gating strategies as a practical mechanism for controlling the cost of LLM inference while retaining meaningful coverage of anomalies. These

contributions are validated on two publicly available benchmark datasets, BGL and HDFS, through automated verification, retrieval-aware faithfulness metrics, and human evaluation.

The remainder of this article is organized as follows: Section 2 discusses the related work. Section 3 presents the proposed Screener-Reasoner methodology. Section 4 reports the empirical evaluation on BGL and HDFS. Section 5 concludes this article and outlines directions for future research.

2. RELATED WORK

Research [22,24] has shown that the performance of anomaly detection is affected by the imperfection of using log parsers. Without using a log parser, both NeuralLog [22] and LogLLM [25] employ a BERT tokenizer (WordPiece) to process raw logs. NeuralLog embeds the alphabetic units of the content in raw log sequences, thereby learning anomalies from these embedded vectors. LogLLM embeds log sequences, projects them into vectors compatible with Llama (LLM), and packages them as prompts to enable the LLM to identify anomalies. ContextLog [26] does not use a log parser either. Instead, ContextLog uses a GPT-supported tokenizer, BPE (Byte Pair Encoding) [27], to embed both alphabetic and non-alphabetic units from the header and content parts of raw log sequences, thereby preserving all contextual information for anomaly detection.

The nature of the log dataset reveals a clear imbalance between normal and anomalous classes. Class resampling methods must be considered to achieve unbiased classification. Ma et al. [28] conclude that random oversampling of the minority class (anomalous) is the optimal class balancing strategy for log-based anomaly detection.

Due to its efficient parallel processing and strong ability to retain long-term information within input sequences, the multi-head self-attention mechanism serves as the current standard for anomaly detection. The computational complexity of self-attention is $O(n^2)$, where n is the token sequence length in a log session. In log-based anomaly detection, to retain all contextual information, it must include both alphabetic and non-alphabetic tokens from embedded log messages. This causes the sequence length of a log session to grow to thousands of tokens, making the self-attention mechanism inefficient for such long sequences. Different self-attention variants have been developed to reduce the computational complexity from quadratic to linear,

thereby enhancing the efficiency of self-attention in handling long input sequences. Instead of computing attentions for all tokens, Sparse Transformer [29] leverages sparsity in the attention mechanism. Using sparse patterns to do factorized attention, such as striding, restricts each token’s attention to a subset of tokens, reducing computational complexity to $O(n\sqrt{n})$ but still capturing both local and long-range dependencies. Reformer [30] develops Locality-Sensitive Hashing (LSH) attention to group similar tokens into buckets and limit attention computation within these buckets, reducing the computation complexity of self-attention to $O(n\log n)$. Wang et al. [31] demonstrate that a low-rank matrix can approximate the self-attention mechanism and propose a linear transformer, Linformer, which reduces the computational complexity of self-attention to $O(n)$ in both time and space.

Within information retrieval, BM25 [32] is a widely used probabilistic ranking method for document retrieval. Robertson et al. [32] discuss BM25 within the Probabilistic Relevance Framework and show that its scoring function estimates the relevance between a query and a document by combining term frequency, inverse document frequency, and document length normalization. In log retrieval, these properties are especially suitable because related anomalous sessions often share recurring lexical cues, such as component names, exception terms, subsystem identifiers, and error phrases. BM25 provides a lightweight, training-free retrieval mechanism for matching a target anomalous session to similar historical anomaly sessions and normal contrast sessions, supplying the external evidence required for pattern-match and contrast claims.

Evaluating only the quality of the generated output does not provide a complete assessment of retrieval-augmented generation. Es et al. [33] proposed RAGAs as a reference-free evaluation framework for retrieval-augmented generation pipelines and characterized evaluation along three dimensions: faithfulness, context relevance, and

answer relevance. This perspective is directly relevant to the proposed framework because the Reasoner also relies on retrieved evidence to generate explanations. However, unlike open-domain question answering, the target task in this work is structured log anomaly explanation with explicit evidence identifiers and line-level evidence spans. This work adopts the evaluation perspective of RAGAs and adapts selected metrics to the log-analysis setting, enabling the assessment of both retrieval and evidence-grounded explanation quality under structured, machine-verifiable constraints.

3. Methodology

The proposed framework, shown in Figure 3, includes three closely connected components: a parser-free Screener for anomaly detection, a Reasoner for retrieval-based explanation generation, and a Gating layer for budget-conscious deployment. The Screener classifies each log session as normal or anomalous. But a binary label alone does not reveal which log lines triggered the alarm, why the observed behavior is anomalous, or how the incident relates to previously seen failure modes. The proposed framework bridges this explanation gap by pairing the Screener with a Reasoner that generates structured, evidence-grounded explanations for every screener-flagged anomaly. In this combination, the proposed framework preserves the lightweight Screener's efficiency while introducing an expressive explanation layer only for suspicious sessions. This design is important in practice because a large log stream may contain tens of thousands of sessions, but only a small fraction is predicted to be anomalous. The more expensive reasoning process is reserved for the sessions that require further inspection.

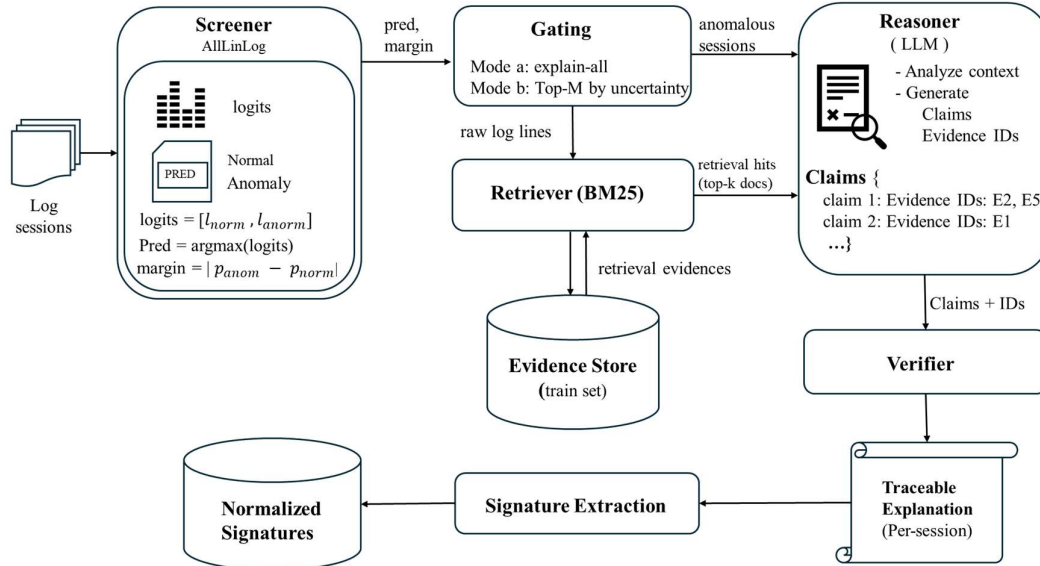


Figure 3. Overview of the Screener-Reasoner Framework

3.1 Screener

The proposed Screener is designed as a lightweight, log-based anomaly detector.

It is composed of three major components:

- 1) Data Processing Layer: directly embedding raw log sessions using the GPT-4 BPE tokenizer to preserve all semantic and temporal information.
- 2) Linear Self-Attention (LSA) Encoder: reduces the computation complexity of self-attention to enable the proposed model to handle long log sequences efficiently.
- 3) Classifier: aggregating the encoder outputs and performing the overall anomaly detection process.

Figure 4 illustrates the Screener workflow, including data processing, the encoder, and classification stages. The following subsections present the methods of the proposed Screener.

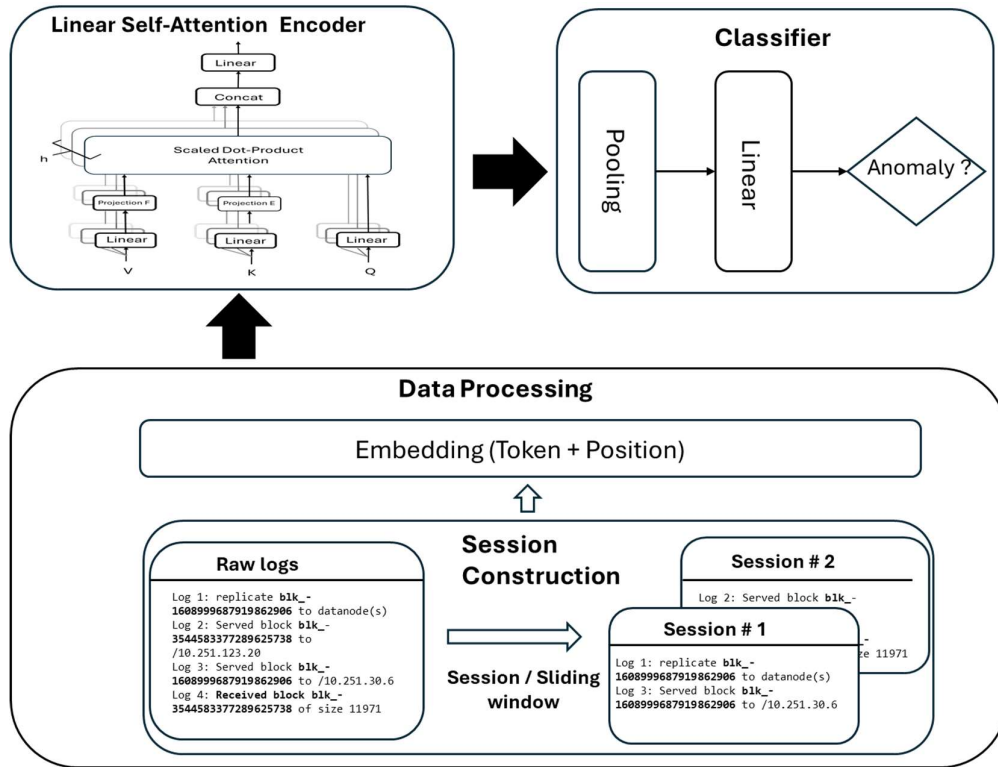


Figure 4. Workflow of the proposed Screener

3.1.1 Data Processing

3.1.1.1 Session Construction

Research typically uses open-source system log datasets, such as BGL and HDFS, to evaluate the performance of log-based anomaly detection models [34]. Without relying on a complex log parser, the proposed Screener uses raw logs directly as input, making it lightweight and robust for handling evolving software system logs.

System logs are typically a continuous, unstructured stream of textual entries that contain application sessions. A sequence of log messages forms a session either based on the same session identifier or a fixed-size window. As illustrated in Figure 5 and Figure 6, the proposed model employs two different session construction methods depending on the nature of the log dataset.

(a) Session ID Grouping (Figure 5)

For datasets such as HDFS, where logs can be linked by a unique identifier (e.g., block IDs), log entries are grouped into sessions based on the shared identifier. Logs referring to the same block ID are grouped into a session.

(b) Sliding Window Grouping (Figure 6)

For BGL that lacks clear identifiers, we use a sliding-window approach to generate sessions. As shown in Figure 6, each session consists of a fixed number of consecutive log entries. For example, with a window size of three and a step size of three, non-overlapping sessions are created from the raw log stream.

Let $L = \{l_1, l_2, \dots, l_N\}$ be the log messages. Define $S = \{s_1, s_2, \dots, s_M\}$ is a collection of sessions created from set L . A session is segmented as:

$$S_m = \{l_{m1}, l_{m2}, \dots, l_{mw}\} \text{ where } w = \text{window size} \quad (1)$$

The proposed Screener is a supervised deep learning model that requires labels for

classification. A session is the basic unit for processing. To assign the label (normal or anomalous) to a session, the rule is that if any log in a session contains an anomalous marker (e.g., does not start with ‘-’ in BGL), the entire session is labeled as anomalous. This session-based input construction enables the Screener to capture temporal dependencies and semantic relationships across logs within a session.

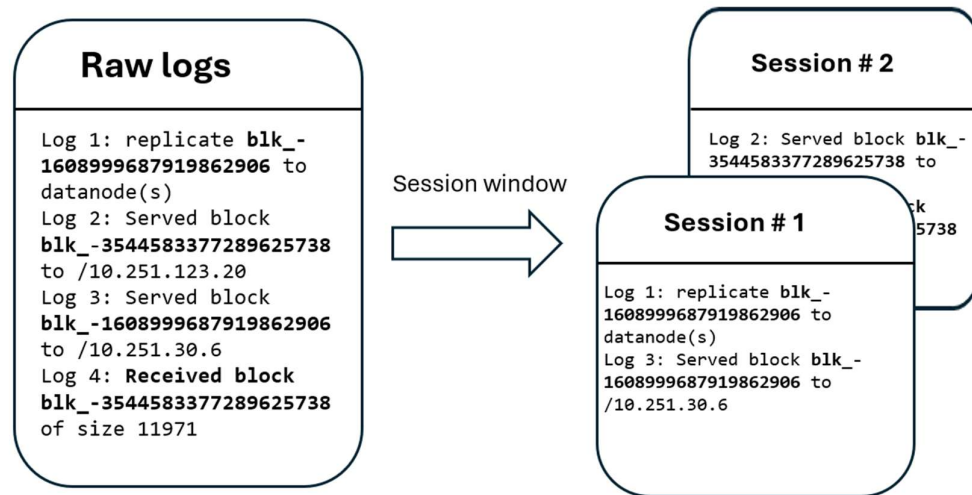


Figure 5. Grouping sessions by session identifiers, such as block ID.

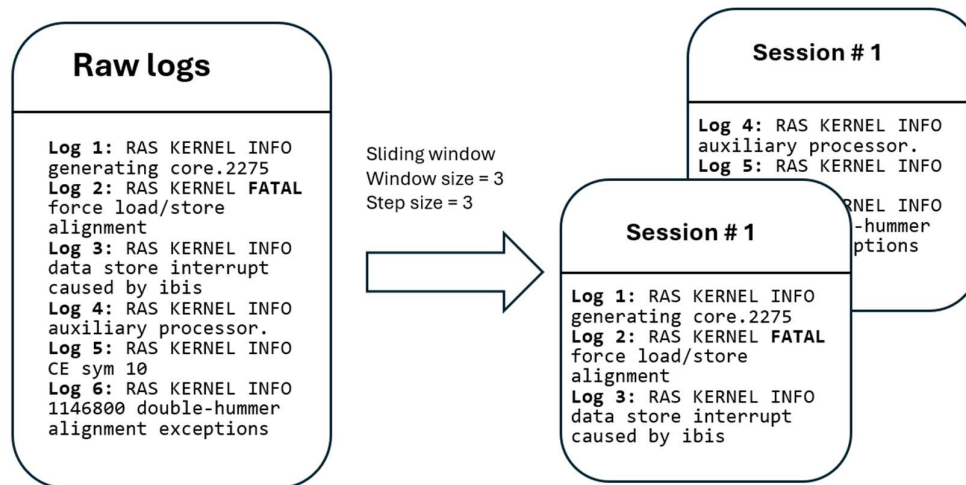


Figure 6. Constructing sessions uses a fixed-size sliding window over the raw system log.

3.1.1.2 Log Tokenization and Vectorization

After constructing session logs, these unstructured log messages need to be

converted into structured vectors for input to the encoder. The proposed Screener uses GPT-4 BPE, a byte-level subword tokenizer, to split raw log messages into tokens and then encode these tokens into IDs, as shown in Figure 7.

BPE [27], a method for tokenizing text, first converts the text into a byte array. It repeatedly merges the most frequent byte pairs to create new subword units until the desired vocabulary size is reached. Because it tokenizes text at the byte level, each character, including special characters and emojis, is represented by a single byte. The proposed model utilizes GPT-4 BPE to address Out-Of-Vocabulary (OOV) issues by breaking unseen words into subword tokens, which are then embedded into vectors within the segmented sessions, making them compatible with the encoder's input. As in Figure 7, the unseen word “crond” is split into “cr” and “ond”. Let T be the pretrained GPT-4 tokenizer. The session s_m is tokenized by T to yield a sequence of token IDs.

$$x_m = T(s_m) = [t_1^{(m)}, t_2^{(m)}, \dots, t_{n_m}^{(m)}], t_i^{(m)} \in \mathbb{Z}_+ \quad (2)$$

```
Raw Log      : crond pam_unix 16448 : session closed for user root
Split to Tokens : ['cr', 'ond', 'pam', '_unix', ' ', '164', '48', ' ', 'session', ' closed', ' for', ' user', ' root']
Encode to IDs  : [5192, 2159, 42290, 81672, 220, 10513, 2166, 551, 3882, 8036, 369, 1217, 3789]
Decode to Log  : crond pam_unix 16448 : session closed for user root

Raw Log      : ERROR: Disk failure at sector 4096.
Split to Tokens : ['ERROR', ' ', ' Disk', ' failure', ' at', ' sector', ' ', '409', '6', ' ']
Encode to IDs  : [3767, 25, 39968, 8060, 520, 10706, 220, 12378, 21, 13]
Decode to Log  : ERROR: Disk failure at sector 4096.
```

Figure 7. System log tokenized by GPT-4 BPE

Table 1: Average and maximum token counts in dataset sessions

Dataset	# of log messages	Grouping method	# Avg of tokens of a session	# Max of tokens of a session
BGL	4,747,963	Sliding window=10	756	2756
		Sliding window=20	1453	4841
		Session window (same blk-id)	1006	15983
HDFS	11,175,629			

The proposed Screener segments tokenized log messages into log sessions based on dataset characteristics. Table 1 shows the session lengths of BGL and HDFS. In the BGL dataset, there are 4,747,963 log messages. When a window size of 20 is used to form a session, the average token length of a session is 1,453, and the maximum token length is 4,841. For HDFS, the average session token length is 1,006 and the maximum length is 15,983. This long input sequence length will be a challenge for the traditional Transformer encoder. The proposed model addresses this issue with an efficient encoder, as stated in Section 3.1.2.

3.1.1.3 Position Embeddings

To incorporate temporal information when evaluating token importance in parallel using the Transformer's self-attention mechanism, a position embedding must be added to the input sequence. To enable deployment of the proposed Screener on low-resource IoT or edge devices, the counting-based absolute position embedding is selected for its computational efficiency, and no external libraries are required. The position IDs are created as $p_m = [0, 1, \dots, n_{m-1}]$, then maps them into a learnable vector with the exact

dimensions as the token vector. The final input matrix $E_m \in \mathbb{R}^{n_m \times d}$ for each session m is computed as follows:

$$E_m = Embed_{token}(x_m) + Embed_{position}(p_m) \quad (3)$$

where:

$$Embed_{token} : \mathbb{Z}_+^{n_m} \rightarrow \mathbb{R}^{n_m \times d}$$

$$Embed_{position} : \mathbb{Z}_+^{n_m} \rightarrow \mathbb{R}^{n_m \times d}$$

n_m : the sequence length of session m

d : the embedding dimension, we set $d = 128$ in the proposed model

3.1.1.4 Handling Data Imbalance with Oversampling

A significant class imbalance is present in the datasets, as shown in Table 2. Setting the window size (w) to 20 results in an anomalous session ratio of 9% for BGL. Using the block ID, the anomalous session ratio for HDFS is 3%. Given the class imbalances in system logs, it is essential to consider class resampling to achieve unbiased performance in log anomaly detection.

Table 2: The ratio of anomalous sessions of BGL and HDFS datasets

Dataset	# of Log messages	# of Log sessions	# Anomalous sessions	Anomaly ratio
BGL (w size=20)	4,747,963	237,389	20,711	0.09
HDFS	11,175,629	575,061	16,838	0.03

The class resampling techniques can be grouped into three categories: oversampling, undersampling, and hybrid [28]. From a data perspective, they can also be divided into resampling from raw data and resampling from the feature space (where the raw data is embedded). When resampling is done directly on raw data, the data distribution should be adjusted before feature embedding. Specifically, this involves either duplicating minority-class samples or selectively removing majority-class samples from the original log sequences.

The proposed Screener employs Random Oversampling (ROS) on the raw data. If β is the target proportion of the anomalous class for oversampling in a dataset, and α is the current proportion of the anomalous class with $\alpha < \beta$, the total number of log sessions to classify is N . To reach the proportion of β , the training data will need to be randomly sampled from the training set as an additional anomalous session quantity X , which satisfies $\alpha \times N + X = \beta(N + X)$. The following formula can be used to calculate X .

$$X = \frac{N(\beta - \alpha)}{1 - \beta} \quad (4)$$

For BGL, the current percentage of the anomalous class is 0.09 (9%) for α . The target percentage for the anomalous class β is set to 0.4 (40%) in the proposed

screening implementation.

The random oversampling function O applies to session set S to rebalance the minority class S' . Let α be the original minority class ratio, β the expected minority ratio, and S_1 be the set of anomalous sessions.

$$S' = O(S; \alpha, \beta) \text{ where } \alpha = \frac{|S_1|}{|S|} \quad (5)$$

After incorporating the oversampled anomalous samples, the number of anomalous sessions can be calculated as follows:

$$|S_1^{new}| = \left| \frac{\beta(1-\alpha)}{1-\beta} \cdot |S| \right| \quad (6)$$

3.1.2 Linear Self-Attention Encoder

3.1.2.1 Reducing Self-Attention Complexity

The self-attention function maps a query and a collection of key-value pairs to produce the attention output. The queries, keys, values, and outputs are represented as vectors. The output is computed as a weighted sum of the values, with weights derived from a compatibility function that evaluates the query against its corresponding key. Scaled dot-product self-attention is illustrated in Figure 8, where the input originates from the same source and is duplicated as Q (Query), K (Key), and V (Value) with the same dimension d . The dot products between the query and each key are calculated, then each is scaled by the square root of the dimension. Then, the softmax function is applied to produce weights for the values. The output matrix is determined as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V \quad (7)$$

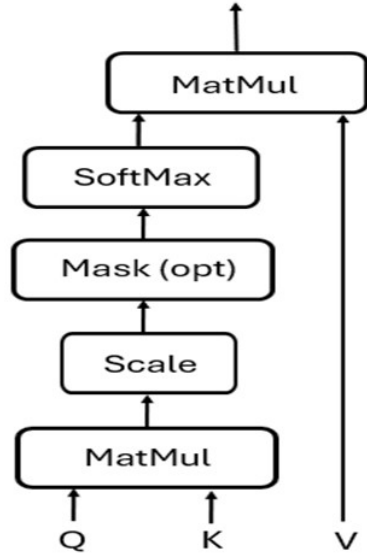


Figure 8. Scaled dot-product self-attention

Instead of applying a single attention function directly to the keys, values, and queries of dimension d_{model} , it is more effective to project the queries, keys, and values separately through h different learned linear projections, mapping them to d_k , d_k , and d_v dimensions, respectively. After these projections, the attention function is applied in parallel to transform the queries, keys, and values, producing output values of dimension d_v . These output values are then concatenated and projected once more to produce the result. Therefore, the multi-head self-attention encoder can simultaneously attend to different aspects of the input by attending to multiple representation subspaces at various positions. In contrast, using a single attention head limits the encoder's expressiveness, as it tends to average out diverse relationships, reducing its ability to capture complex dependencies effectively. This expression describes the multi-head attention:

$$Multihead(Q, K, V) = Concat(head_1, \dots, head_h) W^O \quad (8)$$

$$\text{where } head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

The projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$,

$W_i^V \in \mathbb{R}^{d_{model} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{model}}$

However, the self-attention mechanism requires each token to attend to every other token in the sequence, resulting in quadratic complexity in both time and memory. This becomes a performance bottleneck for long log sequences, as observed in the log session preparation.

The linear self-attention [31] approach addresses quadratic complexity by leveraging the low-rank nature of the attention matrix. Full-rank self-attention can be accurately approximated using substantially fewer dimensions. Instead of computing full attention over the $n \times n$ space, the linear self-attention introduces two learnable projection matrices in the multi-headed self-attention, as shown in Figure 9.

$E \in \mathbb{R}^{k \times n}$ used to compress the key sequence

$F \in \mathbb{R}^{k \times n}$ used to compress the value sequence

These matrices reduce the quadratic attention computation to the following linear form:

$$Linear - Attn(Q, K, V) = softmax\left(\frac{Q(EK)^T}{\sqrt{d}}\right)(FV) \quad (9)$$

This reduces the complexity from $O(n^2)$ to $O(nk)$, where $k \ll n$.

The computational complexity of the standard Transformer and linear self-attention is shown in Table 3, where n is the input sequence length, d is the model embedding dimension, and k is a constant with $k \ll n$.

Table 3: The computational complexity of the transformer and linear self-attention

operation	Transformer complexity	Linear self-attention complexity
Computing QK^T	$O(n^2d)$	$O(nkd)$
Softmax	$O(n^2)$	$O(nk)$
Multiplying by V	$O(n^2d)$	$O(nkd)$
Total Complexity	$O(n^2)$	$O(n)$

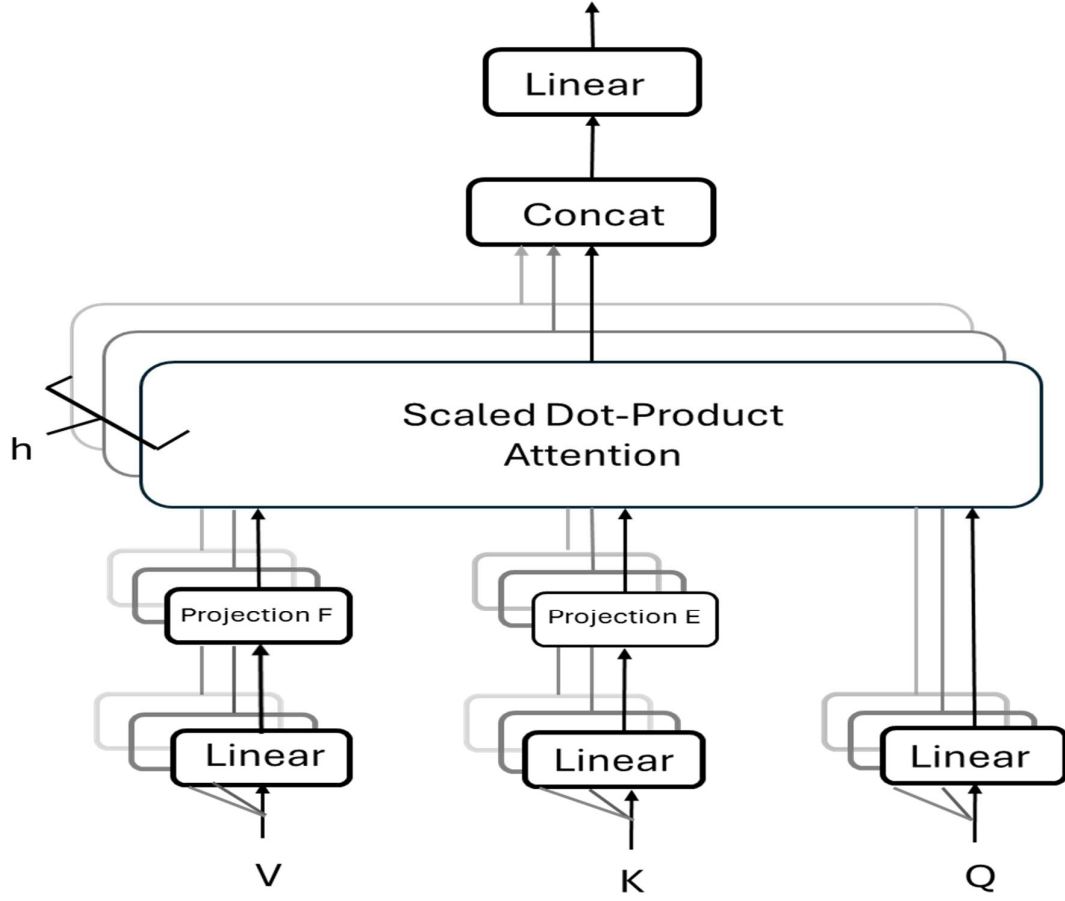


Figure 9. Multi-head self-attention with projection E and F

3.1.2.2 Efficient Encoder

The proposed Screener reduces the quadratic computation complexity by using a linear self-attention encoder. As illustrated in Figure 9, it allows the encoder to compute a contextual attention matrix with linear time and memory complexity in the sequence length. The encoder projects the original key (KW_i^K) and value (VW_i^V) layers, which are $n \times d$ in dimension, into $k \times d$ dimensional projected key and value layers using learnable matrices E and F . Then, the encoder applies the scaled dot-product and the softmax function to Q ($n \times d$) and K ($k \times d$) to compute the attention score. With this attention score and V ($k \times d$), it obtains the attention matrix for each head as follows:

$$\begin{aligned}
head_i &= \text{Attention}(QW_i^Q, E_iKW_i^K, F_iVW_i^V) \\
&= \text{softmax}\left(\frac{QW_i^Q(E_iKW_i^K)^T}{\sqrt{d_k}}\right) \cdot F_iVW_i^V
\end{aligned} \tag{10}$$

To make the encoder lighter, the proposed Screener adopts a layer-wise parameter-sharing technique utilizing a single projection matrix E , with $E_i = F_i = E$ across all layers and heads. This reduces the number of additional parameters in the proposed model, consequently decreasing its memory footprint.

3.1.3 Classification Layer

The encoder processes the input sequence and produces the output $H \in \mathbb{R}^{n \times d}$, which is then passed to the classification layer to perform anomaly detection. The classifier uses average pooling across the sequence length dimension to get the session-level representation as follows:

$$h_{avg} = \frac{1}{n} \sum_{i=1}^n H_i \tag{11}$$

where $H_i \in \mathbb{R}^d$ is the encoded representation of the i -th token in the sequence. To perform the classification, the pooled representation $h_{avg} \in \mathbb{R}^d$ is then passed through a linear layer:

$$z = W \cdot h_{avg} + b \tag{12}$$

Where $W \in \mathbb{R}^{2 \times d}$ and $b \in \mathbb{R}^2$ are all learnable parameters, and $z \in \mathbb{R}^2$ represents the unnormalized logits for the two classes: normal and anomalous.

3.2 Reasoner

The Reasoner is built on a retrieval-augmented generation (RAG) architecture in which a larger language model (LLM) is guided by both retrieved evidence and a machine-verifiable output schema. The proposed Reasoner consists of several closely

connected sub-components: evidence store, retriever, prompt builder, LLM inference, verifier, and signature generator. First, the evidence store provides a searchable corpus of normalized historical sessions. Next, the retriever selects the most relevant evidence documents for the target anomalous session. These documents and the target session are assembled into a fixed prompt format that guides the LLM to generate a structured explanation. After generation, the verifier automatically checks whether the output satisfies the required schema and whether its claims are properly grounded in the cited evidence. The generated anomaly signature is finally normalized to support aggregation and downstream analysis.

The Reasoner is designed for forensic explanations rather than root-cause diagnosis. Its goal is to summarize the abnormal features of a session, connect them to previously observed anomaly patterns, and, when appropriate, compare them with normal evidence. Every generated claim must be supported by explicit evidence identifiers and evidence spans, enabling automated verification and human review to ensure traceability.

3.2.1 Evidence Store

The evidence store is a pre-computed corpus of normalized log sessions taken from the train split. It provides the factual basis upon which the LLM must base every explanatory claim. It is constructed based on two requirements: (1) only train-split sessions are indexed to prevent data leakage from the test set, and (2) both normal and anomalous sessions are included to enable the LLM to produce a contrast claim that highlights behavioral differences between anomalous and normal log sessions.

Raw log messages contain dynamic parameters, including IP addresses, UUIDs, hexadecimal memory addresses, file system paths, and numerical counts. These dynamic portions vary across sessions without changing the underlying behavior

pattern. If left unnormalized, lexical retrieval would match incidental parameter values instead of structural log patterns. The proposed framework uses a dataset-specific log-normalizer that replaces each dynamic parameter with a typed placeholder. Let l denote a raw log line and $N(.)$ the normalizer. The normalized line is computed as:

$$l' = N(l) = \text{reg_chain}(l, P_{\text{dataset}}) \quad (13)$$

Where P_{dataset} is an ordered list of regular expression substitution rules specific to the dataset. For BGL, the rule set includes node identifiers, hexadecimal addresses, file system paths, and numeric values. For HDFS, an analogous rule set normalizes block identifiers, IP addresses, and port numbers. All rules are applied in a fixed order to avoid interaction effects.

For datasets like HDFS, where anomalous and normal sessions mostly share the same vocabulary and differ mainly in event sequences, lexical normalization alone is not enough for effective retrieval. The proposed framework, therefore, augments each evidence document with a structural summary consisting of a set of operation-count tags and anomaly-indicative structural flags. For each HDFS session, the normalizer counts the occurrence of key operations and appends a structural anomaly tag (e.g., INCOMPLETE_PIPELINE, EXCESS_REPLICATION) when characteristic event patterns are detected. These injected tokens serve as discriminative features in BM25 [32], enabling the retriever to distinguish structurally anomalous sessions from normal ones that otherwise share the same vocabulary.

The evidence store produces one evidence document per session. Given a set of training sessions $S_{\text{train}} = \{s_1, s_2, \dots, s_m\}$. For each session s_i , the normalized text is computed by applying $N(.)$ to all lines in the session, and concatenating their structural summary if any:

$$d_i = N(s_i) \parallel \text{structural}(s_i) \quad (14)$$

Each document is stored as a tuple (e_i, d_i, m_i) , where e_i is a unique evidence identifier, d_i is the normalized text, and m_i is a metadata record containing the session label, dataset name, line count, and normalization statistics. The label in the metadata is used only for retrieval routing to assemble anomaly examples and normal contrasts. Exposing the label to the LLM is intentional. It enables the model to distinguish anomalous examples from the normal contrast session, which is necessary to generate correctly typed claims. This does not constitute label leakage because the labels refer to reference evidence sourced from the train split, not to the query session under analysis. The schema of an evidence document in the evidence store is shown in Figure 10.

EvidenceDoc	
evidence_id	"E_BGL_03700010"
session_id	"BGL_03700010"
evidence_type	"session"
text (BM25 indexed)	10 normalized lines, joined by \n Example: <pre>"<IPV6> <NUM>. <NUM> <NODE> RAS KERNEL INFO 1 torus receiver x- input pipe error(s) (dcr <HEX>) detected\n<IPV6> <NUM>..."</pre> Replacements: <pre>IP address -> <IPV6> numbers -> <NUM> node id -> <NODE> hex value -> <HEX></pre>
metadata	<pre>label : 0 (0=normal, 1=anomaly) dataset : "BGL" num_lines : 10 original_length : 1,757 (chars) normalized_length: 1,677 (chars) param_stats : { NODE: 20, IPV6: 10, HEX: 9, NUM: 103 }</pre>

Figure 10. Schema of an evidence document in the evidence store

At inference time, the evidence store provides up to five pieces of evidence per query session. Following this framework's naming convention, the query session itself

is designated as E_0 . The top-4 retrieval anomaly examples are labeled E_1 through E_4 , and the highest-scoring normal session is labeled E_5 . This fixed six-slot evidence layout enables the trace schema described in Section 3.2.3 to enforce that every claim references a specific evidence identifier and its corresponding line-level span. With this, the reasoner establishes a verifiable chain of evidence between the explanation and the raw log data.

3.2.2 Retriever

The retriever selects the most relevant subset of documents from the evidence store to serve as the grounding context for the LLM to generate explanations after the Screener flags a query session. The retrieval step is critical. Without external evidence, an LLM can only reference the query session itself, precluding pattern-match and contrast claims. The retrieval acts as the conduit through which historical failure patterns and normal behavior baselines are injected into each explanation prompt.

The retriever in the proposed framework uses BM25 [32] to rank evidence documents with respect to the query session:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/avgdl)} \quad (15)$$

Where q is the query session, $f(t, d)$ is the term frequency of token t in document d , $IDF(t)$ is the inverse document frequency, $|d|$ is the document length in tokens, and $avgdl$ is the average document length over the corpus. $k_1 \in \mathbb{R}^+$ control term-frequency saturation, and $b \in [0, 1]$ controls document-length normalization. The proposed framework sets k_1 to 1.5 and b to 0.75. BM25 is especially suitable for log retrieval for two reasons. First, log messages feature highly repetitive structured vocabulary, including component names, error codes, and kernel identifiers. This results in strong term overlap between related failure events. Second, BM25 requires

no additional training beyond the normalization used during evidence creation. At index build time, each evidence document is tokenized by lowercasing and splitting on whitespace. The BM25 index is then constructed over the resulting corpus of N token lists, where N is 332,356 for BGL and 402,542 for HDFS.

The retriever constructs each query session by joining its log lines with newline delimiters, appending the structural summary output if available (HDFS only), and passing the result through the same dataset normalizer used during evidence construction. The normalizer replaces instance-specific identifiers, such as node IDs, IP addresses, hex values, and integers, with typed placeholders, ensuring that sessions recording the same failure on different nodes produce identical query tokens. The retriever implements a mixed-evidence strategy that returns two conceptually distinct subsets, corresponding to the evidence slots presented in the prompt: (1) Anomaly examples E_1 to E_4 , the four highest-scoring documents among training sessions labeled as anomalous. These give the LLM previously seen failure patterns similar to the current query, supporting pattern-matching claims. (2) Normal contrast E_5 : the highest-scoring document labeled as normal from the training session. This serves as a representative baseline of normal behavior for the same subsystem or workload type.

The query session is designated E_0 and is never retrieved for the store. The identifiers E_0 through E_5 propagate directly into the prompt, where each cited line is addressed by the fine-grained notation E_i-L_j , which represents evidence slot i , and line j . Four anomalous and one normal evidence allocation were chosen to provide sufficient diversity for pattern-match grounding while reserving one slot for contrast evidence.

3.2.3 Prompt Builder

The prompt builder assembles the query session and the retrieved evidence into a two-part prompt that the LLM uses to generate a structured, machine-verifiable explanation. Let E_0 denote a query session, $\{E_1, \dots, E_5\}$ the retrieved evidence, $\hat{p}$ the anomaly probability, and m be the confidence margin produced by the Screener. The prompt P is constructed as:

$$P = (P_{sys}, P_{user}) \quad (16)$$

$$P_{sys} = \text{Template}(D, S) \quad (17)$$

$$P_{user} = \text{Format}(E_0, \{E_1, \dots, E_5\}, \hat{p}, m) \quad (18)$$

Where D is the dataset configuration, and S is the output specification defined below.

The system prompt P_{sys} defines the analytic context: dataset identity, component vocabulary (e.g., KERNEL and APP for BGL; DATANODE and NAMENODE for HDFS), the E_i - L_j span notation introduced in section 3.2.2, the three required claim types (observation, pattern_match, and contrast), the signature naming convention, and an annotated output example that specifies expected fields and constraints. The user prompt P_{user} instantiates session-specific content: the session identifier, $\hat{p}$ and m , the line-numbered E_0 log block, and the E_1 to E_5 evidence blocks. The structural constraints embedded in P are designed to be directly enforceable by the verifier described in Section 3.2.4.

The E_0 log lines are passed through the same dataset normalizer used during evidence construction. The normalizer replaces instance-specific identifiers, such as block IDs, with typed placeholders, so that E_0 and E_1 through E_5 share a consistent lexical representation, allowing the LLM to focus on structural log patterns rather than incidental parameter differences. All log lines in E_0 and E_1 through E_5 are prefixed with their span identifiers, and a header declares the total line count and valid

span range for each evidence slot, which bounds the line references the LLM may cite. For HDFS, the structural summary described in Section 3.2.1 is appended to E_0 so that the LLM observes the same operation-count tag present in the retrieved evidence. Each E_1 through E_5 block additionally carries a metadata header that includes the evidence label (anomaly or normal). Exposing the label to the LLM is intentional. It enables the model to distinguish anomalous examples from the normal contrast session, which is necessary to generate correctly typed claims. This does not constitute label leakage because the labels refer to reference evidence sourced from the train split, not to the query session E_0 under analysis.

The prompt specifies a structured output format, which is referred to as the trace schema. The LLM must produce a structured explanation T , defined as follows:

$$T = (\hat{y}, \text{summary}, \sigma, C) \quad (19)$$

where $\hat{y} \in \{\text{anomaly}, \text{normal}\}$ is the predicted label, *summary* is a concise forensic description with quantified observations. $\sigma = (\text{name}, \{E_i\}_{\text{matched}})$ is the anomaly signature with matched evidence identifiers. $C = \{c_1, c_2, c_3\}$ is a set of exactly three claims. Each claim is a tuple:

$$c_k = (\tau_k, \text{text}_k, \mathcal{E}_k, \mathcal{R}_k) \quad (20)$$

where $\tau_k \in \{\text{observation}, \text{pattern_match}, \text{contrast}\}$ is the claim type, text_k is the claim text, $\mathcal{E}_k \subseteq \{E_0, \dots, E_5\}$ is the set of referenced evidence identifiers, and $\mathcal{R}_k$ is a non-empty set of evidence spans in E_i-L_j format. The prompt enforces a claim-type completeness constraint requiring exactly one claim of each type:

$$\{\tau_1, \tau_2, \tau_3\} = \{\text{observation}, \text{pattern_match}, \text{contrast}\} \quad (21)$$

This constraint ensures that every explanation integrates direct observation from E_0 , pattern matches against anomaly examples E_1 through E_4 , and behavioral contrasts with the normal session E_5 .

The system prompt further specifies formatting rules that make it machine-

verifiable. Each element in $\mathcal{R}_k$ must conform to either the single-line format E_i-L_j format or the range format E_i-L_j to E_i-L_k . Bare evidence IDs, alternative separators, and placeholder line numbers are not allowed. Every claim must reference at least one span within the specified valid range, and the claim text must mention terms from the cited lines to ensure span-level verifiability. The assembled prompt pair (P_{sys}, P_{user}) is submitted to the LLM with JSON-mode output enabled. The returned JSON response is parsed into the explanation T structure and then sent to the verifier for schema and evidence grounding validation.

3.2.4 Verifier

The structural constraints of the prompt builder guide the LLM toward well-formed output, but prompt-time instructions alone cannot guarantee faithfulness. The LLM may fabricate evidence identifiers that do not exist in the provided context. The cited log lines may contain no anomaly-related content. The produced span reference may exceed the actual line count of an evidence slot. The verifier addresses this gap by using rule-based, post-hoc validation of each explanation T against its corresponding evidence, without requiring an additional LLM call. The verification function is defined as:

$$V(T, E_0, \{E_1, \dots, E_5\}, \mathcal{M}) \rightarrow (b, \mathcal{F}) \quad (22)$$

where $\mathcal{M}$ maps slot identifiers to evidence store identifiers, $b \in \{pass, fail\}$ is the overall verdict, and $\mathcal{F} = \{f_1, \dots, f_n\}$ is a set of individual check outcomes, each carrying a status $s_i \in \{PASS, FAIL, WARN\}$. The overall verdict follows strict failure semantics:

$$b = pass \Leftrightarrow \forall f_i \in \mathcal{F}, s_i \neq FAIL \quad (23)$$

The WARNING outcomes are recorded as soft quality signals but do not block acceptance, preventing the pipeline from discarding explanations that are

substantively correct but exhibit minor stylistic deviations.

The rule-based verifications fall into three categories: (1) Structural checks ensure that the explanation conforms to the trace schema where required fields are present, every referenced evidence identifier belongs to the provided evidence set, the fraction of claims citing at least one evidence source meets a minimum coverage threshold, and the anomaly signature follows the canonical `COMPONENT__ERROR__TYPE` format. (2) Grounding checks determine whether each claim is supported by the evidence it references, at two levels of detail: first at the document level (claim keywords are found somewhere in the cited evidence) and second at the span level (keywords appear specifically within the cited E_i - L_j lines). The span-level check is more rigorous and aims to identify claims that cite the correct evidence ID but point to irrelevant lines. (3) Severity checks address the most frequent hallucination pattern observed in practice: the LLM often cites standard INFO log lines as if they were error indicators. This verification ensures that at least one cited line in E_θ contains a valid severity keyword, such as ERROR, WARN, FATAL, or a dataset-specific exception term. If none contains such keywords, the explanation is marked as a probable hallucination.

3.2.5 Signature Generator

Aggregating thousands of LLM-generated explanations into a coherent anomaly landscape requires a consistent labeling scheme. The signature generator assigns each explanation T a standardized label in `COMPONENT__ERROR__TYPE` format (e.g., `KERNEL_DATA_TLB_ERROR`, `DATANODE_BLOCK_WRITE_FAILURE`), enabling deduplication, frequency counting, and cross-session pattern analysis. Instead of choosing from a fixed taxonomy, the LLM produces a raw signature name σ_{raw} as part of its output, guided by dataset-specific component vocabularies and

example names provided in the system prompt. A deterministic normalization step then maps the raw name to a canonical form:

$$\sigma(T) = \text{Normalize}(\sigma_{\text{raw}}(T)) \quad (24)$$

This step is essential because, without canonicalization, the same failure mode can fragment into lexical variants that undermine exact-match aggregation. For instance, the LLM may generate `KERNEL__FATAL__DATA_TLB_ERROR` (with an unnecessary severity segment, `FATAL`) or `KERNEL__FATAL__data TLB error interrupt` (verbose free-text) for what should automatically be `KERNEL__DATA_TLB_ERROR`. The normalizer removes the severity label added by the LLM, despite clear instructions to leave it out, consolidates different phrasings into standard error types, and fixes component prefixes. Signature format compliance is also enforced by the verifier (Section 3.2.4) as a soft warning.

3.3 Gating

The Reasoner pipeline invokes an LLM for each session that the Screener marks as anomalous. In datasets with high anomaly frequencies, LLM costs increase proportionally. Much of this may be redundant when many sessions share the same underlying failure mode. The Gating module sits between the Screener and the Reasoner and controls which anomalous session receives LLM explanations under a given budget constraint. Let N denote the number of sessions predicted as anomalous and $B \in (0,1]$ a budget ratio specifying the fraction to explain. The gating function selects a subset $\mathcal{G} \subseteq \{x_1, \dots, x_n\}$ with $|\mathcal{G}| = \lfloor B \cdot N \rfloor$:

$$\mathcal{G}(B) = \text{top-K}(\{x_i\}, u(x_i), K = \lfloor B \cdot N \rfloor) \quad (25)$$

where $u(x) = 1 - m(x)$ is the Screener uncertainty and $m(x) = |p_{\text{anom}}(x) - p_{\text{norm}}(x)|$ is the classification margin. Sessions with low margin are those where the Screener is least certain and are prioritized for explanation. This is based on the idea

that high-confidence anomalies are more likely to align with well-understood failure patterns, whereas borderline cases benefit most from detailed LLM analysis. When budget B is set to 1, the gating reverts to the explain-all baseline, which serves as the default mode and reference point for assessing the cost-quality trade-off in Section 4.

4. EXPERIMENTS

A series of experiments is conducted to evaluate the proposed Screener-Reasoner framework across two dimensions: (1) detection accuracy of the Screener, and (2) explanation quality, faithfulness, and cost efficiency of the Reasoner pipeline. The evaluation is structured around three research questions. RQ1 determines whether the framework generates traceable, evidence-grounded explanations that meet both automated verification and human judgment. RQ2 measures the contribution of retrieval-augmented evidence grounding by comparing it to a no-retrieval baseline. RQ3 analyzes the cost-quality trade-off under confidence-based gating at different budget levels. Sections 4.1 and 4.2 present the Screener’s detection performance, including comparisons with baseline models and an ablation study of architectural options. Sections 4.3 to 4.6 sequentially address the three research questions. All experiments are performed on two public log datasets, BGL and HDFS, using an Intel Core i9-13900HX CPU at 2.20 GHz, 32 GB of RAM, and an NVIDIA GeForce RTX 4080 GPU with 12 GB of memory, running Ubuntu 24.04 LTS with PyTorch 2.5.1.

4.1 Benchmark Datasets

4.1.1 Public Datasets and Splitting Strategy

In our experiments, publicly available open-source log datasets are used to evaluate the performance of the proposed Screener and to compare it with baseline anomaly detection models. The adopted datasets are BGL and HDFS [34]. Table 1 summarizes these datasets. The BGL dataset is a publicly available collection of logs from a Blue Gene/L supercomputer system at Lawrence Livermore National Laboratory (LLNL) in Livermore, California. The HDFS (Hadoop Distributed File System) dataset

is publicly available and was generated by running Hadoop-based MapReduce jobs on over 200 Amazon EC2 nodes.

The dataset is divided into training (70%), validation (15%), and test (15%) sets. The training set was used to train the model; the validation set was used for hyperparameter tuning and early stopping; and the final performance was reported on the unseen test set to assess generalization.

4.2 Screener

For the proposed Screener configuration, the minority class oversampling ratio is set to 0.4. The k value for linear self-attention is set to 32. The embedding dimension is set to 128. To retain all feature information, the input sequence is not truncated. The maximum sequence length of the proposed model is set to the maximum length encountered during the construction of input sessions.

4.2.1 Loss Function & Class Weighting

The proposed model adopts a weighted cross-entropy loss function to address the class imbalance commonly observed in log datasets. During training, the weighted loss function assigns a higher penalty to misclassified anomalous instances, helping to prevent bias toward the normal class. Let $\mathbf{z} = [z_0, z_1] \in \mathbb{R}^2$ represent the logit vector output by the classification layer, $y \in \{0,1\}$ be the ground truth label, and the class weight vector $\mathbf{w} = [w_0, w_1]$. The loss for a single training sample is defined as:

$$\mathcal{L}(\mathbf{z}, y) = -w_y \cdot \log \left(\frac{e^{z_y}}{\sum_{j=0}^1 e^{z_j}} \right) = -w_y \cdot \left(z_y - \log \sum_{j=0}^1 e^{z_j} \right) \quad (26)$$

, where w_y denotes the weight assigned to the class.

In the experiment, the weight $w_0 = 1.0$ is assigned to the normal class, and the

weight $w_l = 4.0$ is assigned to the anomalous class. This approach suggests that the model focuses more on the anomalous class to reduce the bias caused by the skewed distribution of the normal class in the training set.

4.2.2 Optimizer & Learning Rate Schedule

The proposed model employs the AdamW optimizer [35], which decouples the weight decay term from gradient updates, ensuring consistent and effective regularization. The optimizer is configured with a weight decay of 10^{-2} and an epsilon of 10^{-8} . The initial learning rate is 5×10^{-4} and scheduled by ReduceLROnPlateau scheduler with the factor set to 0.1

4.2.3 Baseline Comparison

4.2.3.1 Evaluation metrics

To evaluate Screener performance, this work uses widely adopted metrics: Precision, Recall, and F1-score. Let True Positive (TP) be the number of anomalies that the model correctly detects. False Positive (FP) is the number of normal sessions mistakenly identified as anomalies. True Negative (TN) refers to the number of normal sessions correctly identified as normal. False Negative (FN) is the number of anomalous sessions the model fails to recognize.

Precision denotes the percentage of correctly predicted anomalies over all anomalous predictions. $Precision = \frac{TP}{TP + FP}$

Recall represents the percentage of correctly predicted anomalies over all real anomalies. $Recall = \frac{TP}{TP + FN}$

F1-score shows the harmonic mean of the Precision and the Recall.

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

4.2.3.2 Comparison with Baseline Detection Models

The performance metrics of the proposed Screener are compared with those of the following models on the HDFS and BGL public datasets, as shown in Table 4. Those results, except AllLinLog, are cited from FSMFLog [11]. DeepLog [6], LogCluster [20], LogAnomaly [36], SwissLog [37], PLELog [38], and FSMFLog [11] all employ a log parser to transform unstructured system logs to structured data. AllLinLog is the only one that does not utilize log parsing. The proposed model achieves the highest F1 Score on both the HDFS and BGL datasets.

Table 4: Comparison of various methods on public datasets

Methods	HDFS			BGL		
	Precision	Recall	F1-score	Precision	Recall	F1-score
DeepLog	0.879	0.812	0.844	0.823	0.807	0.815
LogCluster	0.851	0.745	0.794	0.578	0.829	0.681
LogAnomaly	0.965	0.834	0.895	0.859	0.912	0.885
SwissLog	0.960	0.885	0.921	0.941	0.852	0.894
PLELog	0.923	0.734	0.818	0.911	0.811	0.858
FSMFLog	0.981	0.952	0.966	0.972	0.951	0.961
AllLinLog	0.997	0.995	0.996	0.999	0.999	0.999

4.2.4 Screener Ablation Study

To evaluate the effectiveness and efficiency of the Screener, an ablation study is conducted on the BGL dataset, focusing on three architecture configurations: the low-rank approximation factor k , the number of linear self-attention encoder layers, and a comparison with the standard transformer encoder in terms of detection performance and inference time. All evaluations are performed using the same data preprocessing

pipeline, loss function configuration, and optimizer settings mentioned in section 4.1.

(A) Effect of Low Rank Approximation Factor k

The linear self-attention encoder replaces full self-attention with a low-rank approximation to address the quadratic complexity of computation time and storage requirements. To assess the optimal value for k , anomaly detection is performed on BGL dataset with k in range of $\{4, 8, 16, 32, 64, 128\}$ while keeping all other configurations of model as constants which include the model embedding dimension set as 128, the layer of linear self-attention encoder set as one and the number of heads set as 4. The result is shown in Figure 11, which reveals that the optimal value of k is 32, with an F1-score of 0.99888.

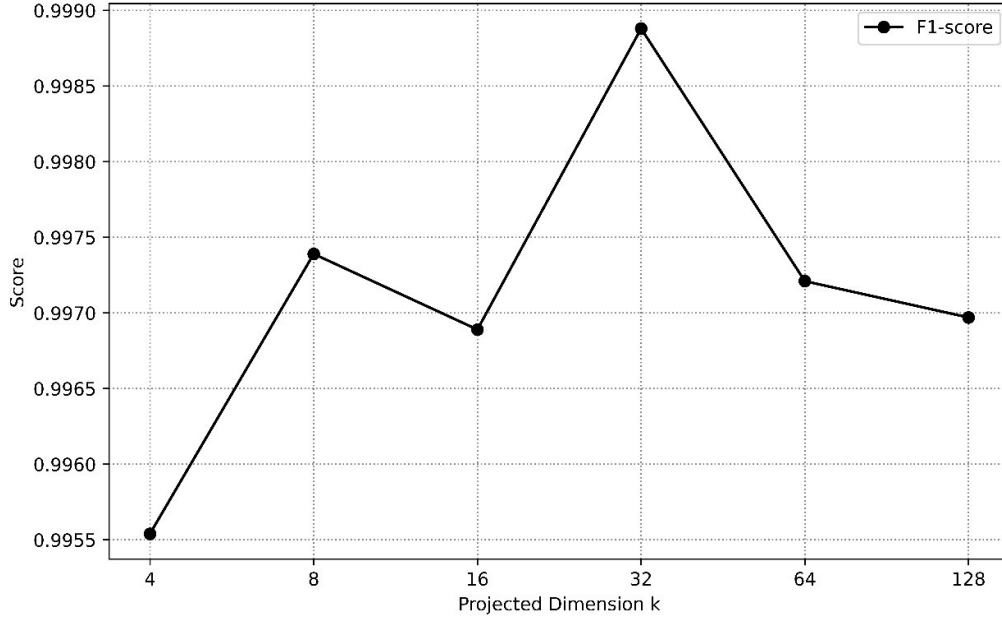


Figure 11. The effect of different low-rank approximation factor k

(B) Effect of Linear Self-Attention Layer Depth

The Screener can stack multiple linear self-attention encoder layers to form a complex anomaly detection framework. To identify the best configuration for log-based anomaly detection tasks, the Screener is tested with different layer setups using the BGL

dataset. The evaluation starts with a single encoder layer with 4 attention heads, then increases to 4 encoder layers with 32 heads, keeping other model parameters unchanged. Based on the results shown in Figure 12, the optimal number of encoder layers is one, with an F1-score of 0.99888.

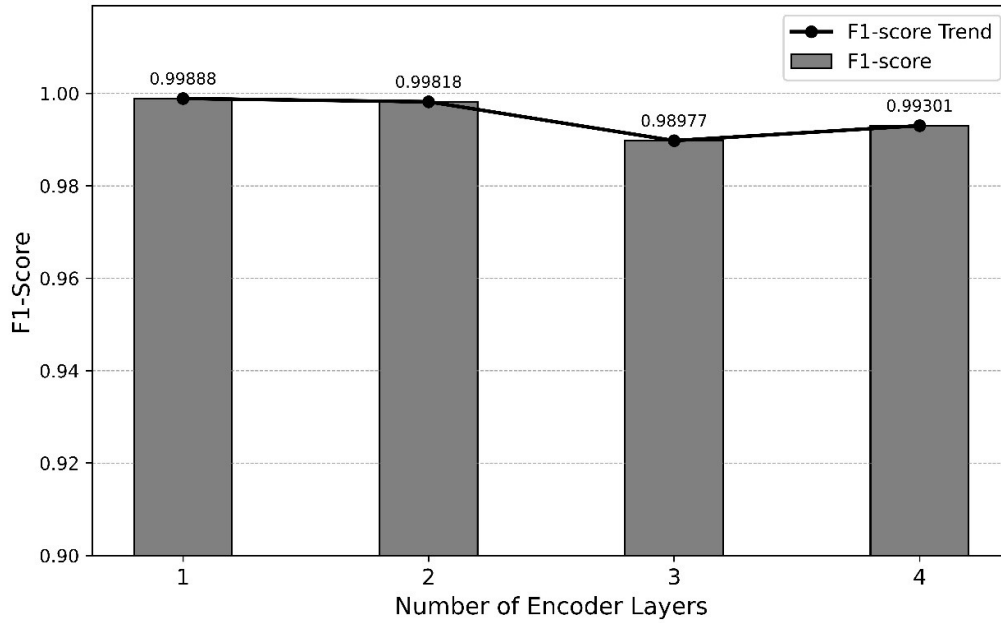


Figure 12. Impact of encoder layer depth on F1-score

(C) Comparison with Standard Transformer Encoder

The proposed Screener employs a linear self-attention encoder to reduce the computational complexity of full self-attention from quadratic to linear complexity by incorporating a low-rank approximation. To evaluate the cost incurred by the low-rank approximation, the encoder is replaced with a standard Transformer encoder, and the detection performance of the two encoders is compared on the BGL dataset. From the results shown in Figure 13, the linear self-attention encoder reduces computational complexity at the cost of a 0.079% drop in F1-score compared to the standard Transformer encoder (from 0.99967 to 0.99888).

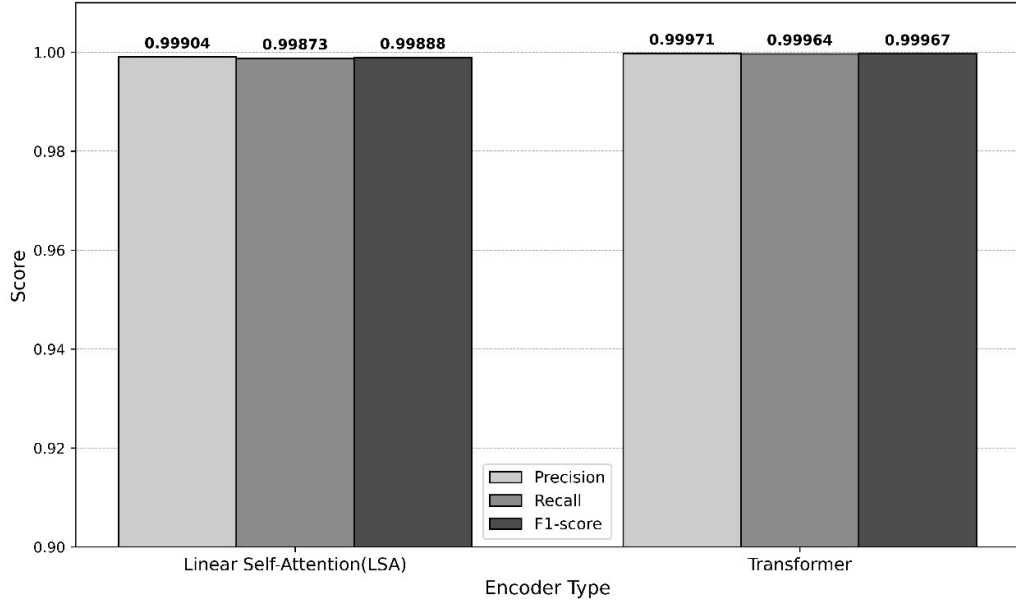


Figure 13. Comparison of anomaly detection performance between the linear self-attention encoder and the transformer encoder

On the other hand, as Table 5 illustrates, the linear self-attention encoder is faster and more efficient than the Transformer encoder. It achieves significant complexity reductions in both inference time and memory consumption. This study begins with a batch size of 1 for real-time anomaly detection, then increases it to 16 to simulate devices that collect more log sessions and perform anomaly detection in batches. For measuring memory usage, `torch.cuda.max_memory_allocated` is used to determine the peak memory allocation. This is because PyTorch, by design, pre-allocates and preserves memory during operation, so the memory delta between before and after inference cannot be measured. For the inference time benchmark, inference is executed 100 times, and the average time is computed.

Table 5: Benchmark analysis: transformer encoder vs. linear self-attention (LSA) encoder (Batch size 1 & 16)

Encoder	Batch size	Peak Memory used	Avg Inference Time	Throughput
Transformer	1	893.22 MB	4.7541 ms	210.35 samples/sec
LSA	1	709.09 MB	0.8143 ms	1228.07 samples/sec
Transformer	16	3824.70 MB	51.2782 ms	312.02 samples/sec
LSA	16	911.70 MB	3.9677 ms	4032.52 samples/sec

From Table 6, the linear self-attention encoder outperforms the Transformer encoder in terms of memory efficiency. Table 7 also shows that the linear self-attention encoder scales inference time more efficiently: the Transformer encoder’s inference time increases by approximately 10.8 times when the batch size is increased to 16. The linear self-attention encoder’s inference time only grows by about 4.9 times, indicating that it handles large batches more efficiently. Furthermore, Table 8 shows that the linear self-attention encoder achieves 12.9 times the throughput of the Transformer encoder at a batch size of 16. These results indicate that the Transformer encoder struggles with large batches due to its quadratic computational complexity of self-attention. In contrast, the linear self-attention encoder demonstrates significantly higher efficiency due to its linear computational complexity.

Table 6: Memory efficiency comparison between transformer encoder and linear self-attention (LSA) encoder

Encoder	Batch Size	Memory Growth (1 $\rightarrow$ 16)
Transformer	1 $\rightarrow$ 16	893.22 MB $\rightarrow$ 3824.70 MB (+2,931.48 MB)
LSA	1 $\rightarrow$ 16	709.09 MB $\rightarrow$ 911.70 MB (+202.61 MB)

Table 7: Inference time efficiency comparison between transformer encoder and linear self-attention (LSA) encoder

Encoder	Batch Size	Inference Time Increase (1 $\rightarrow$ 16)
Transformer	1 $\rightarrow$ 16	4.7541 ms $\rightarrow$ 51.2782ms (10.8x increase)
LSA	1 $\rightarrow$ 16	0.8143 ms $\rightarrow$ 3.9677ms (4.9x increase)

Table 8: Throughput comparison between transformer encoder and linear self-attention (LSA) encoder

Encoder	Batch Size	Throughput (samples/sec)
Transformer	1 $\rightarrow$ 16	210.35 $\rightarrow$ 312.02
LSA	1 $\rightarrow$ 16	1228.07 $\rightarrow$ 4032.52

4.3 Reasoner

4.3.1 LLM configuration

The Reasoner pipeline uses GPT-5.1 as its core language model, accessed through the OpenAI chat-compatible API with JSON mode enabled to ensure structurally valid output. The sampling temperature is set to 0.1 to promote deterministic responses while maintaining minimal randomness to prevent repetitive output. Each request includes the full system prompt (including dataset context,

component vocabulary, JSON schema, and evidence block E_0 through E_5) and the user prompt, which contains the query session. The LLM produces a single-turn explanation with a maximum of 2048 completion tokens and a 120-second timeout.

4.3.2 Evaluation Metrics

Reasoner quality is measured along the complementary axes of automated verification and human evaluation. In automated verification, the Verification Pass Rate (VPR) is the fraction of explained sessions whose outputs pass all nine verifier checks without any failures. Because VPR is binary per session, it captures worst-case compliance; a single unsupported claim or missing signature is enough to mark the session as failed. We also report evidence utilization, defined as the fraction of retrieved evidence slots (E_1 through E_5) explicitly cited by at least one claim, and signature coverage, the proportion of training-derived anomaly pattern cards that appear in the full run output across all test sessions. For the cost dimension, we record total token consumption, average tokens per session, and LLM latency at the mean and 95th percentile.

Human evaluation complements the automated metrics on a stratified random sample of 50 sessions per dataset. A domain expert evaluates each explanation based on three 5-point criteria: correctness (factual accuracy of the anomaly diagnosis), completeness (whether all anomaly signals in the logs are included), and evidence-grounding (whether the cited evidence adequately supports claims). A binary, actionable label is also assigned to indicate whether an engineer can act on the explanation without reviewing the raw logs. These four dimensions directly address RQ1 and provide independent validation of the automated verification process.

4.4 RQ1: Explanation Quality

4.4.1 Automated Verification Results

Table 9 summarizes the Reasoner pipeline results on both datasets. The pipeline explains all Screener-flagged anomaly sessions: 5,850 for BGL and 2,527 for HDFS, achieving a 100% verification pass rate on both datasets. Every explanation passes all verifier checks without any failures. This result confirms that combining JSON-mode structured output with the evidence-grounded prompt design described in section 3.2 consistently produces valid, citation-supported explanations at scale. The average token usage per session is 6,496 for BGL and 8,120 for HDFS. The average LLM latency is 7.6 seconds for BGL and 8.1 seconds for HDFS, with the 95th percentile below 14.4 seconds for both datasets.

Table 9: Reasoner pipeline results on BGL and HDFS

Metric	BGL	HDFS
Anomalies Explained	5,850	2,527
Verification pass rate	100%	100%
Unique signatures	237	98
Avg tokens/session	6,496	8,120
Mean latency (seconds)	7.6	8.1
P95 latency (seconds)	13.2	14.4

4.4.2 Human Evaluation

Human evaluation results on a stratified random sample of 50 sessions per dataset are shown in Table 10. Across both datasets, correctness scores exceed 4.9 out

of 5, and completeness is nearly perfect (5.00 for BGL and 4.96 for HDFS), showing that the generated explanations accurately identify the root cause and cover all anomaly signals in the logs. Evidence-grounding scores are slightly lower: 4.8 for BGL and 4.64 for HDFS because HDFS sessions sometimes consist only of brief INFO-level messages, which offer less lexical content for the LLM to base its claims on the cited evidence. All 100 sampled explanations are rated as actionable, meaning an engineer can act on each without consulting the raw logs.

Table 10: Human evaluation results (5-point scale, n = 50 per dataset)

Dimension	BGL (mean $\pm$ std)	HDFS (mean $\pm$ std)
Correctness	4.94 $\pm$ 0.24	4.92 $\pm$ 0.27
Completeness	5.00 $\pm$ 0.00	4.96 $\pm$ 0.20
Evidence grounding	4.80 $\pm$ 0.40	4.64 $\pm$ 0.52
Actionable	100%	100%

4.4.3 Anomaly Signature Analysis

The signature generator identifies 237 unique anomaly types across BGL and 98 across HDFS, providing a taxonomy of failure modes. The distribution follows a Zipf-like pattern: a few dominant signatures account for most sessions (for example, `KERNEL__DATA_TLB_ERROR` alone covers 2,337 of 5,850 BGL anomalies, or 39.9%), while the long tail includes 128 singletons in BGL (54%) and 36 in HDFS (36.7%). Figure 14 visualizes this distribution as a rank-frequency plot, and Table 11 lists the five most frequent signatures per dataset. The high singleton rate does not indicate noise. Manual inspection shows these are authentic rare failure modes (e.g., one-time firmware faults or transient network errors) that the LLM correctly identifies

and labels with unique, descriptive signatures. This long-tail discovery offers a practical benefit of the framework: operators can filter ongoing issues by main signature while handling new failures through the singletons.

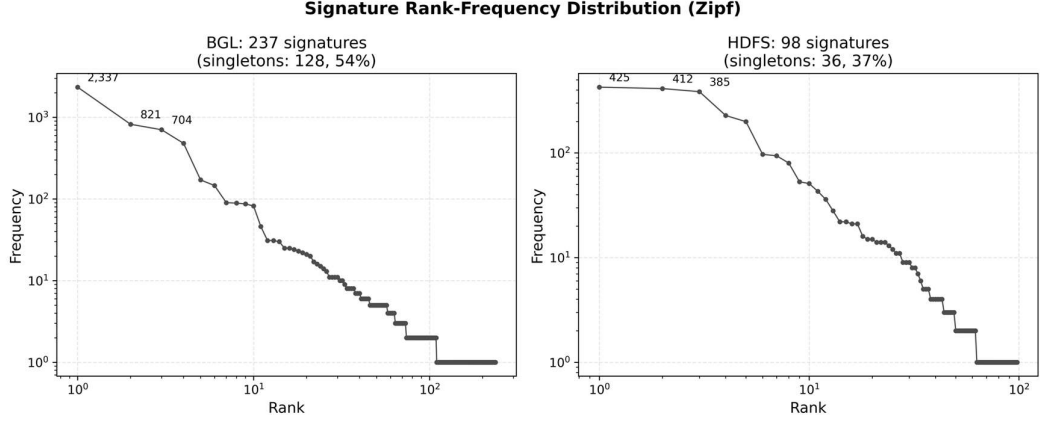


Figure 14. Rank-frequency distribution of anomaly signatures (log scale)

Table 11: Top 5 frequent anomaly signatures per dataset

Rank	BGL signature	Count	HDFS Signature	Count
1	KERNEL__DATA_TLB_ERROR	2,337	DATANODE__BLOCK_WRITE_STREAM_READ_FAILURE	425
2	KERNEL__DATA_STORAGE_INTERRUPT	821	FSDATASET__BLOCK_DELETION_METADATA_MISMATCH	412
3	APP__CIOD_STREAM_ERROR	704	NAMENODE__INCOMPLETE_PIPELINE	385
4	KERNEL__LUSTRE_MOUNT_FAILED	480	FSDATASET__BLOCK_DELETION_METADATA_MISS	228
5	KERNEL__KERNEL_TERMINATED	171	NAMENODE__EXCESS_REPLICATION	199

4.5 RQ2: RAG ablation

4.5.1 Ablation Design

Research question 2 investigates whether retrieval-augmented generation (RAG) significantly enhances evidence grounding in explanations, or whether the LLM can

produce sufficiently grounded explanations solely from the query sessions. This study evaluates this through three complementary analyses.

Two metrics (Faithfulness and Context Precision) are adopted from the RAGAs framework [33] and adapted to the evidence-slot setting of this work, along with a third metric specific to evidence-grounded explanations:

- (1) Faithfulness: the ratio of claims with cited evidence identifiers that are present in the evidence mapping and contain relevant content. A claim is deemed faithful if it cites at least one valid evidence slot from E_1 through E_5 . This serves as a control metric because the combination of JSON-mode generation, evidence-labeled prompts, and a low sampling temperature consistently produces valid citations in a single pass. Faithfulness is expected to be 1.0 regardless of retrieval conditions.
- (2) Context Precision@K ($K = 4$): a rank-aware retrieval quality metric. For each retrieved evidence slot E_1 through E_5 , a binary relevance judgment v_k indicates whether the slot was cited by any claim. Context Precision@K is then:

$$\text{Context Precision@K} = \text{sum}(\text{Precision@k} * v_k) / \text{sum}(v_k) \quad (27)$$

where $\text{precision@k} = \text{sum}(v_k \text{ for } j = 1..k) / k$.

Higher values indicate that the retriever ranks relevant evidence earlier.

- (3) Grounding Breadth: the fraction of claims citing at least one external evidence slot (E_1 through E_4) rather than only the query session (E_0). This is the key metric for RQ2: it measures how well RAG enables the LLM to cross-reference its analysis against known anomaly patterns in the evidence store.

The ablation compares two conditions. In the RAG-on condition, the full pipeline provides E_0 (query session), E_1 through E_4 (BM25-retrieved anomaly sessions), and

E_5 (normal control session) as a prompt. In the no-RAG condition, only E_0 is provided: the retrieval step is skipped, and the prompt contains only the query sessions. Both conditions use the same LLM configuration (section 4.3.1), prompt template, and verification loop. The ablation is performed on a stratified sample that covers all unique anomaly signatures: 338 sessions (237 signatures) for BGL and 245 sessions (98 signatures) for HDFS.

A supplementary analysis compares retrieval results when the BM25 corpus is limited to training data (the production setting) versus expanded to the entire dataset. For 100 randomly selected anomaly sessions per dataset, the top 4 retrieved evidence pieces are compared across both corpus scopes. An overlap rate above 85% confirms that the train-only corpus provides sufficient coverage, showing that the pipeline does not need access to test data for effective retrieval.

4.5.2 Faithfulness and Grounding Impact

The metrics computed on the RAG-on pipeline for all anomaly sessions for BGL and HDFS are reported in

Table 12. Faithfulness is 1.0 across both datasets: every claim in every explanation cites at least one valid evidence slot. This outcome is anticipated since integrating JSON-mode generation, evidence-labeled prompts, and a low sampling temperature reliably produces valid citations in a single pass. Context Precision@4 is 0.991 for BGL and 0.946 for HDFS, indicating that the BM25 retriever consistently ranks relevant evidence in the top positions. Evidence utilization, which is the fraction of available evidence slots cited by at least one claim, is 0.670 for BGL and 0.463 for HDFS. This aligns with dataset-level patterns observed in RQ1: HDFS sessions frequently contain only INFO-level messages, offering less distinctive lexical content for cross-referencing.

Table 12: Baseline metrics (RAG-on) for all anomaly sessions

Metric	BGL (n = 5,850)	HDFS (n = 2,527)
Faithfulness	1.000	1.000
Context Precision@4	0.991 (+ − 0.053)	0.946 (+ − 0.152)
Evidence Utilization	0.670	0.463
Verification Pass Rate	1.000	1.000

Expanding the BM25 corpus from training data to the full dataset caused less than 13% of the top-4 anomaly retrievals to change on either dataset. The top-1 retrieved evidence, which has the highest BM25 relevance score, matches at 88% for BGL and 86% for HDFS. The mean number of new documents introduced by the full dataset corpus is 0.51 (BGL) and 0.31 (HDFS) out of 4 slots. These results confirm that the train-only corpus design is effective: expanding to the entire dataset offers little extra retrieval coverage.

Table 13 presents a direct comparison between RAG-on and no-RAG conditions. Faithfulness remains at 1.0 in both conditions and datasets: the LLM consistently produces valid evidence citations in a single pass, regardless of how many evidence slots are available. Although only E_0 is in the no-RAG condition, the LLM consistently cites E_0 for every claim, and the verifier accepts these citations as valid.

The discriminative metric is Grounding Breadth. With RAG, 66.7% of claims cite external evidence (E_1 through E_4), cross-referencing the query session with known anomaly patterns from the evidence store. Without RAG, Grounding Breadth drops to 0.0%, meaning no claim can cite external evidence because there is none in the prompt. The delta of +0.667 is the same across both datasets, reflecting the prompt

template’s three-claim structure (one observation, one pattern-match, one contrast), where the pattern-match and contrast claim naturally require external evidence.

Furthermore, in the no-RAG condition, 100% of sessions (338 BGL, 245 HDFS) contain at least one hallucinated evidence identifier. Specifically, the contrast claim in each session cites an E_5 slot that does not exist in the evidence mapping. The LLM attempts to reference a normal contrast session even though none was provided. Although the verifier’s keyword-match check still passes (the only available E0 slot satisfies the minimum match-ratio threshold), the structural invalidity of these citations reveals that the LLM attempts cross-referencing even when no external evidence is provided. The absence of retrieved evidence does not suppress the cross-referencing behavior; it merely makes it hallucinatory.

Table 13: RAG-on vs no-RAG ablation results

Metric	BGL	BGL	HDFS	HDFS
	RAG-on	no-RAG	RAG-on	no-RAG
Faithfulness	1.000	1.000	1.000	1.000
Grounding Breadth	0.667	0.000	0.667	0.000
Claims per session	3.0	3.0	3.0	3.0
Hallucinated IDs (sessions)	0	338(100%)	0	245 (100%)

4.6 RQ3: Cost-Quality Trade-off under Gating

4.6.1 Gating Strategy Comparisons

Research question 3 examines whether budget-limited gating, which selects a subset of Screener-flagged anomalies for full LLM explanation, can lower costs while

maintaining coverage of the anomaly signature catalog. Given N anomalous sessions, the gating module selects $K = \text{floor}(B * N)$ sessions for explanation, where B in $(0,1]$ is the budget ratio. Four deployable strategies and one privileged reference strategy are evaluated:

- (1) Random: uniform random sampling without replacement.
- (2) Uncertainty: top-K by $u(x) = 1 - \text{margin}(x)$, where margin is the Screener’s softmax margin between the two probable classes. A higher $u(x)$ indicates lower Screener confidence and, hypothetically, greater information gain from a full LLM explanation.
- (3) Novelty: top-K by $n(x) = 1 - \text{bm25_max}(x) / \text{max}(\text{bm25_max})$, where $\text{bm25_max}(x)$ is the highest BM25 similarity between session x and all documents in the evidence store, and $\text{max}(\text{bm25_max})$ is the maximum bm25_max value across all sessions in the dataset. Sessions with low lexical similarity to the evidence store are assumed to represent novel fault types.
- (4) Combined: top-K by $s(x) = \alpha * u(x) + (1 - \alpha) * n(x)$, mixing uncertainty and novelty with tunable weight α .
- (5) Upper Bound (privileged): greedy selection of one session per signature, starting with the rarest first, until the budget K is exhausted. This strategy is not deployable because it requires knowledge of anomaly signatures, which are only available after LLM execution. The Upper Bound achieves 100% coverage at every tested budget level because the minimum budget $K = \text{floor}(0.1 * N)$ exceeds the number of unique signatures on both datasets (237 for BGL, 98 for HDFS).

The primary evaluation metric is Signature Coverage: the proportion of unique anomaly signatures in the entire corpus that are represented among the selected K sessions. Across all strategies and budget levels on both datasets, every selected

session achieves a Verification Pass Rate of 1.0. This consistent quality outcome demonstrates that gating is simply about balancing coverage and cost. The quality of each session's explanation remains the same regardless of the selection criterion.

Table 14 presents the signature coverage across strategies at three key budget levels. Uncertainty gating outperforms all other deployable strategies when $B \leq 0.5$ on both datasets. At $B = 0.1$, Uncertainty achieves 49.8% coverage on BGL and 55.1% coverage on HDFS, surpassing Random by 22.5 and 13.5 percentage points, respectively. At $B = 0.2$, the advantage widens to 25.4 percentage points (BGL) and 18.9 percentage points (HDFS). Figure 15 visualizes the full coverage-cost Pareto curves for all strategies.

Table 14: Signature coverage (%) by gating strategy and budget (B)

Strategy	BGL	BGL	BGL	HDFS	HDFS	HDFS
	$B = 0.1$	$B = 0.2$	$B = 0.5$	$B = 0.1$	$B = 0.2$	$B = 0.5$
Random	27.3	40.8	67.8	41.6	54.6	77.9
Uncertainty	49.8	66.2	73.0	55.1	73.5	93.9
Novelty	33.3	35.0	35.0	3.1	5.1	22.4
Combined ($\alpha=0.5$)	33.8	35.4	35.4	4.1	6.1	23.5
Upper Bound*	100.0	100.0	100.0	100.0	100.0	100.0

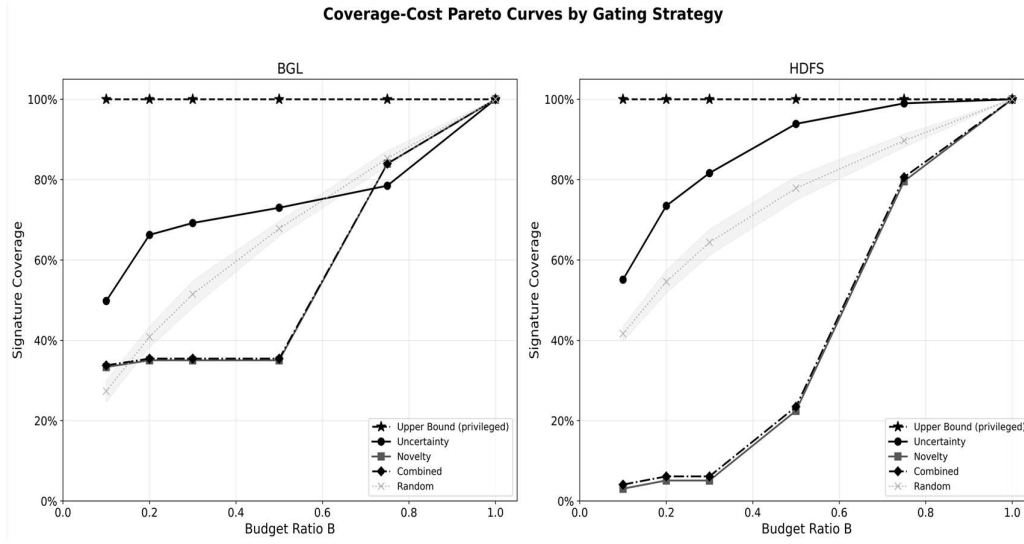


Figure 15. Signature Coverage vs. Budget Ratio Across Gating Strategies

The Novelty strategy underperforms on BGL and performs much worse on HDFS, with only 3.1% coverage when B equals 0.1, compared to Uncertainty's 55.1%. The root cause is low discriminative power in the novelty signal: the coefficient of variation (CV) of the novelty scores $n(x)$ is only 0.29 on BGL and 0.26 on HDFS, meaning that nearly all sessions receive similar novelty values. In contrast, the uncertainty score $u(x)$ yields CV values of 51.84 and 43.79, reflecting variation that is two orders of magnitude greater and effectively differentiating between sessions. The combined strategy inherits this flaw because the mean of $u(x)$ is much lower than that of $n(x)$ in absolute terms. BGL: mean $u(x)$ is 0.0001, mean $n(x)$ is 0.61; HDFS: mean $u(x)$ is 0.0004, mean $n(x)$ is 0.72. Therefore, a weighted combination with the default α of 0.5 is dominated by the novelty term. As a result, the Combined coverage mirrors Novelty across all budget levels, with the optimal mixing weight being alpha equal to 1.0 (Section 4.6.2), where Combined relies solely on Uncertainty.

4.6.2 Budget Sensitivity Analysis

Sweeping the mixing weight α from 0.0 to 1.0 across three budget levels confirms that the optimal weight is 1.0 for both datasets, indicating pure uncertainty with no novelty contribution. Adding any novelty weight monotonically reduces coverage, confirming the coefficient-of-variation analysis in Section 4.6.1. The production gating module, therefore, uses $u(x) = 1 - \text{margin}(x)$ as the sole selection criterion.

With B at 0.20, uncertainty gating detects 66.2% (BGL) and 73.5% (HDFS) of all anomaly signatures, while cutting LLM token usage by 79.6% and 75.4%, respectively. Raising the budget B to 0.50 shows a dataset-dependent trade-off: on BGL, coverage rises just 6.8 percentage points to 73.0%, suggesting a plateau due to the large signature space of 237 types. On HDFS, coverage rises by 20.4 percentage points to 93.9%, nearing completeness considering its smaller signature space of 98 types. The best operating point depends on domain needs: starting with setting B to 0.20, which offers the optimal balance between cost and coverage when broad but incomplete coverage is acceptable, while B set to 0.50 is preferable when near-total signature coverage is required, and the signature space is limited.

4.7 Discussion

The three research questions reveal a layered structure where each finding supports the next. The 100% verification pass rate established in RQ1 is not just an encouraging quality metric; it is essential for the gating analysis in RQ3. Since every explanation passes all verification checks regardless of the sessions selected, the selection problem simplifies to maximizing signature coverage within a token budget without sacrificing model quality. This property arises from three design choices:

JSON-mode structured output, evidence-labeled prompts, and a low sampling temperature (0.1). These combined constraints allow the generation of valid, citation-supported explanations within a single LLM call, without needing retry loops.

RQ2 demonstrated that retrieval-augmented generation is essential, not merely beneficial. Without retrieved evidence, the LLM doesn't just generate less accurate explanations; it produces evidence identifiers that weren't in the prompt, leading to a 100% hallucination rate for E_5 across both datasets. This demonstrates that the cross-referencing embedded in the prompt's three-claim structure is essential to how the model generates responses: it attempts to cite external evidence, regardless of whether such evidence exists. RAG guides this tendency into specific, verifiable evidence slots, transforming a possible weakness (hallucinated citations) into a strength based on trustworthy cross-references. The key practical lesson is clear: any system that skips the retrieval step risks producing explanations that are well-structured but may include fabricated evidence. Relying only on a verifier's keyword-match check cannot reliably detect these issues.

The gating analysis in RQ3 offers a helpful deployment recommendation, but it is specific to the dataset. The uncertainty-gating strategy outperforms all other deployable methods. The limited usefulness of the novelty signal is clear: its coefficient of variation, 0.29 for BGL and 0.26 for HDFS, is two orders of magnitude lower than that of the uncertainty signal, which is 51.84 for BGL and 43.79 for HDFS, respectively. This shows that most sessions have similar novelty scores, causing a systematic bias in top-K selection toward a small group of signatures. The optimal operation point depends on the signature space: small spaces like HDFS, which have 98 types, benefit from budgets up to B equals 0.50, while larger signature spaces, with 237 types for BGL, plateau at B equals 0.20. In both cases, the token savings far outweigh the coverage loss, making the uncertainty-gating strategy a practical

choice for managing production costs.

Several limitations need to be recognized. First, the framework has only been tested with one LLM backbone (GPT-5.1), and its effectiveness with smaller or open-source models remains untested. Second, the two public datasets (BGL and HDFS) concentrate on high-performance computing and distributed storage workloads but do not encompass microservices, network, or security logs. Third, human evaluation relies on a single domain expert rating 50 sessions per dataset, which prevents measurement of inter-rater agreement. Fourth, the anomaly signature taxonomy is created by the LLM itself without any external gold standard for validation. Although manual inspection shows that singletons are truly rare failures (Section 4.4.3), a systematic evaluation of signature quality would bolster this assertion. Finally, the evidence store stays unchanged during evaluation. Temporal drift in log patterns might reduce retrieval accuracy over time, but this effect is not assessed in this study.

5. CONCLUSION

This work presents Screener-Reasoner, a two-stage framework designed for log-based anomaly detection and explanation. The first stage, Screener, uses a linear self-attention encoder along with GPT-4 BPE tokenization to classify log sessions efficiently. This approach eliminates the need for heuristic log parsers and overcomes the out-of-vocabulary limitations faced by template-based methods. The Reasoner stage expands detection into explanations: for each flagged anomaly, a retrieval-augmented LLM generates a structured, evidence-grounded explanation consisting of three claims (observation, pattern match, and contrast), each citing specific evidence slots from the query session and BM25-retrieved reference sessions. An automated verifier enforces structural and factual constraints in a single pass, and a signature normalizer consolidates explanations into a taxonomy of anomaly types.

Experiments on the BGL and HDFS public log datasets show several key findings. The Screener achieves state-of-the-art detection accuracy without relying on log parsing or input truncation, while maintaining all contextual information across sessions of any length. The Reasoner delivers explanations that pass all automated verification tests on both datasets, with human assessments confirming their near-perfect accuracy and full actionability. RAG is structurally essential: in the absence of retrieved evidence, the LLM hallucinates evidence identifiers across all sessions, while RAG grounds two-thirds of all claims with external evidence. Cost control through uncertainty-based gating covers 66% (BGL) and 74% (HDFS) of the anomaly signature catalog with only 20% of the LLM budget, and the optimal operating point varies depending on the size of the signature space.

Multiple options remain available. Testing the framework with smaller or open-source LLM backends could help determine if the single-pass verification property

extends beyond GPT-5.1. Implementing online update mechanisms for the evidence store would address temporal drift in log patterns, which the current static design cannot accommodate. Replacing or supplementing BM25 with dense or hybrid retrievers might enhance retrieval quality in log domains where lexical overlap is less meaningful. Additionally, creating a byte-level tokenization method that segments log messages without splitting parameters and technical terms, such as IP addresses or component names, into subwords would reduce the input sequence length and better preserve the contextual features of system logs for anomaly detection.

REFERENCE

- [1] Roku, Protecting your Roku account, Roku Blog (n.d.).
<https://www.roku.com/blog/protecting-your-roku-account> (accessed February 28, 2025).
- [2] X. Shu, K. Tian, A. Ciambrone, D. Yao, Breaking the Target: An Analysis of Target Data Breach and Lessons Learned, (2017).
<https://doi.org/10.48550/arXiv.1701.04940>.
- [3] M. Chen, A.X. Zheng, J. Lloyd, M.I. Jordan, E. Brewer, Failure diagnosis using decision trees, in: International Conference on Autonomic Computing, 2004. Proceedings., 2004: pp. 36–43. <https://doi.org/10.1109/ICAC.2004.1301345>.
- [4] J. Breier, J. Branišová, Anomaly Detection from Log Files Using Data Mining Techniques, in: K.J. Kim (Ed.), Information Science and Applications, Springer, Berlin, Heidelberg, 2015: pp. 449–457. https://doi.org/10.1007/978-3-662-46578-3_53.
- [5] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, X. Chen, Log clustering based problem identification for online service systems, in: Proceedings of the 38th International Conference on Software Engineering Companion, Association for Computing Machinery, New York, NY, USA, 2016: pp. 102–111.
<https://doi.org/10.1145/2889160.2889232>.
- [6] M. Du, F. Li, G. Zheng, V. Srikumar, DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA, 2017: pp. 1285–1298. <https://doi.org/10.1145/3133956.3134015>.
- [7] S. Lu, X. Wei, Y. Li, L. Wang, Detecting Anomaly in Big Data System Logs Using Convolutional Neural Network, in: 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech), 2018: pp. 151–158.
<https://doi.org/10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.00037>.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is All you Need, in: Advances in Neural Information Processing Systems, Curran Associates, Inc., 2017.
https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fb-d053c1c4a845aa-Abstract.html (accessed June 26, 2025).
- [9] H. Guo, S. Yuan, X. Wu, LogBERT: Log Anomaly Detection via BERT, in: 2021

- International Joint Conference on Neural Networks (IJCNN), 2021: pp. 1–8.
<https://doi.org/10.1109/IJCNN52387.2021.9534113>.
- [10] J. Qi, S. Huang, Z. Luan, S. Yang, C. Fung, H. Yang, D. Qian, J. Shang, Z. Xiao, Z. Wu, LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection, in: 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), 2023: pp. 273–280.
<https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045>.
- [11] W. Niu, Z. Li, Z. He, A. Wang, B. Li, X. Zhang, FSMFLog: Discovering Anomalous Logs Combining Full Semantic Information and Multifeature Fusion, IEEE Internet of Things Journal 11 (2024) 4442–4453.
<https://doi.org/10.1109/JIOT.2023.3300690>.
- [12] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, Q. Tian, LogFormer: a pre-train and tuning pipeline for log anomaly detection, in: Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, AAAI Press, 2024: pp. 135–143. <https://doi.org/10.1609/aaai.v38i1.27764>.
- [13] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, M.R. Lyu, Tools and Benchmarks for Automated Log Parsing, in: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2019: pp. 121–130. <https://doi.org/10.1109/ICSE-SEIP.2019.00021>.
- [14] S. Thaler, V. Menkonvski, M. Petkovic, Towards a neural language model for signature extraction from forensic logs, in: 2017 5th International Symposium on Digital Forensic and Security (ISDFS), 2017: pp. 1–6.
<https://doi.org/10.1109/ISDFS.2017.7916497>.
- [15] Z.M. Jiang, A.E. Hassan, P. Flora, G. Hamann, Abstracting Execution Logs to Execution Events for Enterprise Applications (Short Paper), in: 2008 The Eighth International Conference on Quality Software, 2008: pp. 181–186.
<https://doi.org/10.1109/QSIC.2008.50>.
- [16] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: An Online Log Parsing Approach with Fixed Depth Tree, in: 2017 IEEE International Conference on Web Services (ICWS), 2017: pp. 33–40. <https://doi.org/10.1109/ICWS.2017.13>.
- [17] P. He, J. Zhu, P. Xu, Z. Zheng, M.R. Lyu, A Directed Acyclic Graph Approach to Online Log Parsing, (2018). <https://doi.org/10.48550/arXiv.1806.04356>.
- [18] M. Nagappan, M.A. Vouk, Abstracting log lines to log event types for mining software system logs, in: 2010 7th IEEE Working Conference on Mining

- Software Repositories (MSR 2010), 2010: pp. 114–117.
<https://doi.org/10.1109/MSR.2010.5463281>.
- [19] H. Dai, H. Li, C.-S. Chen, W. Shang, T.-H. Chen, Logram: Efficient Log Parsing Using n-Gram Dictionaries, *IEEE Transactions on Software Engineering* 48 (2022) 879–892. <https://doi.org/10.1109/TSE.2020.3007554>.
- [20] R. Vaarandi, M. Pihelgas, LogCluster - A data clustering and pattern mining algorithm for event logs, in: 2015 11th International Conference on Network and Service Management (CNSM), 2015: pp. 1–7.
<https://doi.org/10.1109/CNSM.2015.7367331>.
- [21] M. Du, F. Li, Spell: Streaming Parsing of System Event Logs, in: 2016 IEEE 16th International Conference on Data Mining (ICDM), 2016: pp. 859–864.
<https://doi.org/10.1109/ICDM.2016.0103>.
- [22] V.-H. Le, H. Zhang, Log-based Anomaly Detection Without Log Parsing, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2021: pp. 492–504.
<https://doi.org/10.1109/ASE51524.2021.9678773>.
- [23] Y. Fu, M. Yan, Z. Xu, X. Xia, X. Zhang, D. Yang, An empirical study of the impact of log parsers on the performance of log-based anomaly detection, *Empir Software Eng* 28 (2022) 6. <https://doi.org/10.1007/s10664-022-10214-6>.
- [24] Z.A. Khan, D. Shin, D. Bianculli, L.C. Briand, Impact of log parsing on deep learning-based anomaly detection, *Empir Software Eng* 29 (2024) 139.
<https://doi.org/10.1007/s10664-024-10533-w>.
- [25] W. Guan, J. Cao, S. Qian, J. Gao, C. Ouyang, LogLLM: Log-based Anomaly Detection Using Large Language Models, (2025).
<https://doi.org/10.48550/arXiv.2411.08561>.
- [26] R. Xiao, W. Li, J. Lu, S. Jin, ContextLog: Non-Parsing Log Anomaly Detection With All Information Preservation and Enhanced Contextual Representation, *IEEE Transactions on Network and Service Management* 21 (2024) 4750–4762.
<https://doi.org/10.1109/TNSM.2024.3400283>.
- [27] R. Sennrich, B. Haddow, A. Birch, Neural Machine Translation of Rare Words with Subword Units, (2016). <https://doi.org/10.48550/arXiv.1508.07909>.
- [28] X. Ma, H. Zou, P. He, J. Keung, Y. Li, X. Yu, F. Sarro, On the Influence of Data Resampling for Deep Learning-Based Log Anomaly Detection: Insights and Recommendations, *IEEE Transactions on Software Engineering* 51 (2025) 243–261. <https://doi.org/10.1109/TSE.2024.3513413>.
- [29] R. Child, S. Gray, A. Radford, I. Sutskever, Generating Long Sequences with Sparse Transformers, (2019). <https://doi.org/10.48550/arXiv.1904.10509>.
- [30] N. Kitaev, Ł. Kaiser, A. Levskaya, Reformer: The Efficient Transformer, (2020).

<https://doi.org/10.48550/arXiv.2001.04451>.

- [31] S. Wang, B.Z. Li, M. Khabsa, H. Fang, H. Ma, Linformer: Self-Attention with Linear Complexity, (2020). <https://doi.org/10.48550/arXiv.2006.04768>.
- [32] S. Robertson, H. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *FNT in Information Retrieval 3* (2009) 333–389.
<https://doi.org/10.1561/15000000019>.
- [33] S. Es, J. James, L. Espinosa Anke, S. Schockaert, RAGAs: Automated Evaluation of Retrieval Augmented Generation, in: N. Aletras, O. De Clercq (Eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, Association for Computational Linguistics, St. Julians, Malta, 2024: pp. 150–158.
<https://doi.org/10.18653/v1/2024.eacl-demo.16>.
- [34] J. Zhu, S. He, P. He, J. Liu, M.R. Lyu, Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics, in: *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 2023: pp. 355–366.
<https://doi.org/10.1109/ISSRE59848.2023.00071>.
- [35] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization, (2019).
<https://doi.org/10.48550/arXiv.1711.05101>.
- [36] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, R. Zhou, LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs, in: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, International Joint Conferences on Artificial Intelligence Organization, Macao, China, 2019: pp. 4739–4745. <https://doi.org/10.24963/ijcai.2019/658>.
- [37] X. Li, P. Chen, L. Jing, Z. He, G. Yu, SwissLog: Robust and Unified Deep Learning Based Log Anomaly Detection for Diverse Faults, in: *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, 2020: pp. 92–103. <https://doi.org/10.1109/ISSRE5003.2020.00018>.
- [38] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, W. Zhang, PLELog: Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation, in: *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2021: pp. 230–231.
<https://doi.org/10.1109/ICSE-Companion52605.2021.00106>.